Naro Fashion — Production Deployment Guide

Complete step-by-step guide with troubleshooting

Version	3.0
Date	October 2026
Author	Rogath J. Urio
Platform	Multi-Tenant SaaS E-Commerce
Stack	Next.js 15 + NestJS 11 + PostgreSQL + Prisma
Audience	The operator / DevOps person who provisions, deploys, rolls back, backs up and troubleshoots production

> *This guide is based on real deployment experience. Every command has been run, every error has been encountered and solved. A new administrator with basic Linux knowledge should be able to deploy the entire platform from scratch using only this document.*
>
> ***Companion runbooks*** *(shorter, kept next to the scripts and updated with them):* `docs/OPS/DEPLOY.md`, `docs/OPS/MIGRATIONS.md`, `docs/OPS/BACKUPS.md`, `docs/OPS/MONITORING.md`. *When this guide and a script disagree,* ***the script wins*** *— read* `deploy.sh` *and* `scripts/ops/*.sh`.

Table of Contents

- What changed in 3.0
- Architecture Overview
- Prerequisites
- Part 1: VPS Provisioning
- Part 2: Server Setup
- Part 3: Application Deployment (first install)
- Part 4: Build Troubleshooting and Production Incidents
- Part 5: PM2 Configuration
- Part 6: Nginx Configuration
- Part 7: DNS Setup
- Part 8: SSL Certificates
- Part 9: Email Setup (Brevo)
- Part 10: Tenant Configuration
- Part 11: CI/CD Pipeline and deploy.sh
- Part 12: Firewall, Security and Backups
- Part 13: Post-Deployment Verification
- Part 14: Known Issues and Solutions
- Part 15: Updating the Application (deploy, rollback, schema changes, data scripts)
- Part 16: Cost Summary
- Appendix A: All Environment Variables Reference
- Appendix B: Quick Reference Commands

What changed in 3.0

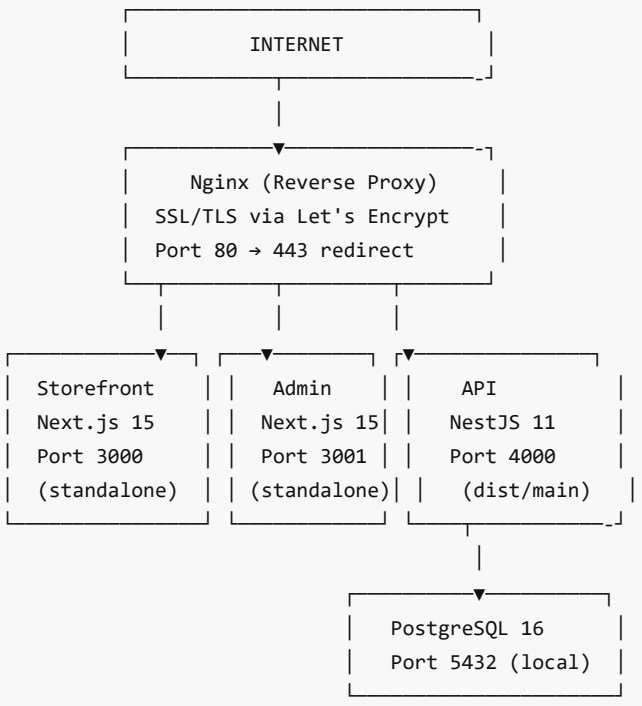
Version 2.0 (April 2026) described a deploy that was `git pull → pnpm install → prisma db push --accept-data-loss → pnpm build → pm2 restart all`. Almost every step of that has since been replaced after real incidents. If you learned the old procedure, **unlearn these**:

Area	2.0 (April 2026) — OBSOLETE	3.0 (October 2026) — CURRENT
CI	Push to `prod` → SSH → `deploy.sh`	Push to `prod` → **`verify` job** (shell checks, frozen install, prisma generate, migrations-vs-schema drift check, `tsc` ×3, API jest, `pnpm audit --audit-level=critical`) → **`deploy` job** (concurrency group, `EXPECTED_SHA`, pinned ECDSA host-key fingerprint)
Branch	`deploy.sh` pulled `origin master` (silently undeployed every push for weeks)	`git reset --hard origin/prod` + assert `HEAD == EXPECTED_SHA`
Dependencies	`pnpm install` (could mutate the lockfile)	`pnpm install --frozen-lockfile`
Schema	`prisma db push --accept-data-loss`	**Committed migrations** + `prisma migrate deploy`, one-time baseline of `0_init`, post-migrate drift guard
Safety net	none	**Pre-deploy `pg_dump`** — deploy aborts if it fails
Build	`pnpm build` (parallel Turbo → OOM on 2GB)	**Sequential, in-place** builds (api → storefront → admin), each after copying the live build aside
Static files	`cp … 2>/dev/null || true` (errors swallowed)	Copied into each standalone server folder, missing `server.js` aborts
Restart	`pm2 restart all` (also restarts other apps on the shared VPS)	`pm2 reload --only naro-api` + hard `pm2 restart --only naro-storefront,naro-admin`, with `GIT_SHA`
Verification	none	Health checks on API `/api/v1/health`, storefront (real tenant `Host`), admin **and one real `/_next/static` asset per app**
Bad release	manual	**Automatic rollback** of builds + source; manual `scripts/ops/rollback.sh --yes`
Backups	ad-hoc crontab `pg_dump | gzip` at 02:00	Nightly `pg-backup.sh` 03:15 UTC (30 days), pre-deploy dumps, **off-site to Cloudflare R2** (rclone + `age` encryption, incl. encrypted server-config bundle), Healthchecks.io alerting, quarterly restore test
Secrets	JWT secrets "recommended"	API **refuses to boot** in production without `JWT_SECRET`/`JWT_REFRESH_SECRET` ≥ 32 chars and different from each other; `TWO_FA_ENCRYPTION_KEY` needed for 2FA

New in this version: Part 4.10–4.17 (the 2026 production incidents), Part 11 (rewritten around the real `deploy.sh`), Part 1.3 and 12.2 (SSH host keys, interception), Part 12.3–12.7 (backups, off-site, restore, server rebuild, restore test), Part 14.8–14.13 (new operational failure modes), Part 15 (deploy / rollback / schema-change / data-script workflow) and Appendix A (complete env reference including the October 2026 variables).

Removed as obsolete: the old `deploy.sh` listing and `.env` backup/restore trick (`git reset --hard` never touched the git-ignored `.env` files), the "manual update" procedure using `git pull origin master` + `pnpm build` + `pm2 restart all` , `prisma db push --accept-data-loss` everywhere, the root-crontab backup recipe, and the `pnpm add multer` step (multer is now a declared dependency of `apps/api`).

Architecture Overview




```
Nightly 03:15 UTC  pg_dump  ──► /var/backups/naro/postgres/ (30 days)
Nightly 04:00 UTC  rclone + age ──► Cloudflare R2 bucket (off-site, encrypted)
Both jobs ping Healthchecks.io (missed / failed run → email to owner)
```

Production URLs:

Domain	Service	Internal Port	PM2 name
`narofashion.co.tz` (+ `www.`)	Storefront (customer shop)	3000	`naro-storefront`
`admin.narofashion.co.tz`	Admin dashboard	3001	`naro-admin`
`api.narofashion.co.tz`	REST API (`/api/v1`)	4000	`naro-api`

> ***The VPS is shared.*** *PM2 on the same box also runs other applications (e.g.* `sportflow-web` *on port 3200, and others under* `/var/www/` *), and Nginx has other sites enabled (* `jumuiya` *,* `kiuruwi` *,* `sportflow` *). Two consequences you must respect:* ***never run*** `pm2 restart all` / `pm2 stop all` / `pm2 delete all` *(always use* `--only <name>` *or the* `naro-*` *names), and* ***never run a Naro deploy while another app is building*** *on the 2GB box (OOM).* `pm2 jlist` *(or* `pm2 list` *) is the source of truth for what is running.*

Prerequisites

Before you begin, you need:

Requirement	Details
Domain name	A registered domain (e.g., `narofashion.co.tz`). For `.co.tz` domains, use Habari Node (habari.co.tz) or any tzNIC-accredited registrar
VPS account	Vultr, Hetzner, DigitalOcean, or Linode account with a credit/debit card
GitHub access	Account with access to `github.com/rjurio/naro-fashion` (or your fork), with rights to manage Actions secrets
SSH client	Git Bash (Windows) or Terminal (Mac/Linux). **Do NOT use PowerShell** for SSH — it has password paste issues
SSH key pair	Ed25519 key pair (generated in the steps below if you do not have one)
A trusted network	SSH from the Bank of Tanzania office network is intercepted (see Part 1.3). Use a mobile hotspot or home connection for anything that touches the server
Email service	Brevo free account (for transactional emails — 300/day free tier)
Backups	Cloudflare account (R2 free tier), Healthchecks.io account (free), and a password manager to hold the `age` private key
Time estimate	First deployment: 3-4 hours. Subsequent deployments: push to `prod`, ~10-20 minutes of CI + build

---

Part 1: VPS Provisioning

1.1 Generate SSH Key (Windows)

IMPORTANT: Use Git Bash, not PowerShell. PowerShell cannot paste passwords into SSH prompts, which will frustrate you during server setup.

Open **Git Bash** and run:

```
# Generate an Ed25519 SSH key
ssh-keygen -t ed25519 -C "your-email@example.com"
# Press Enter to accept default location (~/.ssh/id_ed25519)
# Enter a passphrase (or leave empty for no passphrase)

# Display your public key (copy this for the VPS)
cat ~/.ssh/id_ed25519.pub
```

Copy the entire output (starts with `ssh-ed25519 AAAA...`). You will paste this into your VPS provider.

1.2 Create VPS Server

Option A: Vultr (Recommended — $12/mo)

1. Go to **https://www.vultr.com** and create an account
2. Navigate to **Products** > **Compute** > click **Deploy +**
3. Select **Cloud Compute** > **Shared CPU**

4. Configure:

Setting	Value
Location	Frankfurt, DE (closest to East Africa with good latency)
Image	Ubuntu 24.04 LTS x64
Plan	High Frequency: vhf-1c-2gb (1 vCPU, 2GB RAM, 64GB NVMe) — $12/mo
SSH Key	Click "Add New" and paste your public key from step 1.1
Hostname	`naro-fashion-prod`

5. Click **Deploy Now**
6. Wait 1-2 minutes for the server to provision
7. Note the **IP address** from the server details page (e.g., `80.240.30.107`)

> ***Why Frankfurt?*** *It is the closest major data center region to East Africa. Latency from Dar es Salaam is typically 120-180ms, compared to 200-300ms for US-based servers.*

Option B: Hetzner ($7.50/mo — cheapest)

1. Go to **https://console.hetzner.com**
2. **Note:** Hetzner requires ID verification (passport upload). This can take 1-24 hours.
3. Create Project > Add Server > Ubuntu 24.04, CX22 (2 vCPU, 4GB RAM), Falkenstein
4. Add your SSH key during creation

Option C: DigitalOcean ($12/mo)

1. Go to **https://www.digitalocean.com**
2. Create Droplet > Ubuntu 24.04, Basic, Regular $12/mo, Frankfurt
3. Add your SSH key during creation

1.3 First SSH Connection (verify the host key!)

Use Git Bash (not PowerShell):

```
ssh -o StrictHostKeyChecking=ask root@YOUR_VPS_IP
# Example: ssh root@80.240.30.107
```

On the very first connection SSH shows a fingerprint and asks "Are you sure you want to continue connecting?". **Do not just type "yes".** Compare the fingerprint with the one shown in the provider's web console (Vultr → server → *View Console*, then run `ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub` and `ssh-keygen -lf /etc/ssh/ssh_host_ecdsa_key.pub`).

The production server's keys (verified from the Vultr console on 2026-10-04):

Key type	Fingerprint
ED25519	`SHA256:5/g02liZ1pxrwxbWOHTEGaOtUdrJrtB8uzL7C7bg42M`
ECDSA	`SHA256:2AuvcjqqaC2wpZpD8LVeKiWNgmnS7HybLI+M93wUYxs` (also pinned in the CI deploy job)

Once the key is in `~/.ssh/known_hosts`, always connect with strict checking:

```
ssh -o StrictHostKeyChecking=yes root@80.240.30.107
```

> ***WARNING — corporate network interception.*** *From the Bank of Tanzania office network, SSH to the VPS is intercepted: a different, changing host key is presented (TLS/SSH inspection). If you see* `WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!` *,* ***stop****. Never run* `ssh-keygen -R` *, never accept the new key, never type a password into that session. Disconnect and reconnect from another network (mobile hotspot). A changed key is only legitimate after a server rebuild, and then you re-verify it from the provider console as above.*

If you used a password during VPS creation and cannot paste it:

- In Git Bash, right-click to paste (Ctrl+V does not work in Git Bash)
- Some terminals: use Shift+Insert to paste

1.4 Add SSH Key to Server (if not added during provisioning)

If you created the VPS without an SSH key and are using password authentication:

```
# On the server, paste your public key:
mkdir -p ~/.ssh
echo "ssh-ed25519 AAAA... your-email@example.com" >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

From this point forward, you can SSH without a password. Once key login is confirmed from a trusted network, disable password login (Part 12.2).

Part 2: Server Setup

All commands in this section run **on the VPS as root** (connected via SSH).

2.1 System Update and Essential Packages

```
# Update package lists and upgrade all installed packages
apt update && apt upgrade -y

# Install essential packages
apt install -y \
  curl \
  git \
  build-essential \
  nginx \
  certbot \
  python3-certbot-nginx \
  ufw \
  htop \
  unzip
```

What each package does:

Package	Purpose

`curl`	HTTP client for downloading installers
`git`	Clone the repository
`build-essential`	C/C++ compiler for native Node.js modules
`nginx`	Reverse proxy / web server
`certbot` + `python3-certbot-nginx`	Free SSL certificates from Let's Encrypt
`ufw`	Uncomplicated Firewall
`htop`	Process/memory monitor
`unzip`	Archive extraction

2.2 Install Node.js 22 LTS

```
# Add NodeSource repository for Node.js 22
curl -fsSL https://deb.nodesource.com/setup_22.x | bash -

# Install Node.js
apt install -y nodejs

# Verify
node -v
# Expected output: v22.x.x
```

2.3 Install pnpm and PM2

```
# Install pnpm (package manager — the project uses pnpm workspaces)
# Match the version pinned in package.json "packageManager" (CI uses the same)
npm install -g pnpm@10.12.1

# Install PM2 (process manager — keeps apps running and restarts on crash)
npm install -g pm2

# Verify
pnpm -v    # Expected: 10.12.1
pm2 -v     # Expected: 5.x.x or 6.x.x
```

> *Do **not** install the `prisma` CLI globally. Always use the repo's local Prisma (v6.x) via `pnpm exec prisma` from `packages/database/` — the global v7 CLI has breaking changes.*

2.4 Install PostgreSQL 16

```
# Install PostgreSQL
apt install -y postgresql postgresql-contrib

# Enable and start the service
```

```
systemctl enable postgresql
systemctl start postgresql

# Verify
psql --version
# Expected: psql (PostgreSQL) 16.x
```

2.5 Create Database and User

```
# Generate a strong password (save this — you will need it for .env)
openssl rand -base64 32
# Example output: a1B2c3D4e5F6g7H8i9J0k1L2m3N4o5P6
# SAVE THIS PASSWORD!

# Enter PostgreSQL as the postgres superuser
sudo -u postgres psql
```

Inside the PostgreSQL shell:

```
-- Replace <YOUR_STRONG_PASSWORD> with the password you generated above
CREATE USER naro_admin WITH PASSWORD '<YOUR_STRONG_PASSWORD>';
CREATE DATABASE naro_fashion OWNER naro_admin;
GRANT ALL PRIVILEGES ON DATABASE naro_fashion TO naro_admin;

-- Verify
\l
-- You should see naro_fashion in the list, owned by naro_admin

-- Exit
\q
```

2.6 Generate Secrets

The API **refuses to start in production** (`NODE_ENV=production`) unless `JWT_SECRET` and `JWT_REFRESH_SECRET` are each set, **at least 32 characters**, and **different from each other** (`apps/api/src/auth/util/jwt-secrets.ts`). There is no hard-coded fallback any more. Generate fresh values:

```
# JWT_SECRET (access tokens)
openssl rand -hex 48

# JWT_REFRESH_SECRET (refresh tokens — MUST differ from JWT_SECRET)
openssl rand -hex 48

# TWO_FA_ENCRYPTION_KEY (encrypts admin TOTP secrets — needed to enable 2FA)
openssl rand -hex 32
```

Store all three in your password manager. They go into the `.env` files in Part 3.2.

> `TWO_FA_ENCRYPTION_KEY` ***must NEVER be rotated after any admin has enrolled in 2FA.*** *The TOTP secrets in the database are AES-256-GCM-encrypted with a key derived from it; a new value makes every enrolled admin's 2FA unusable (they would need the manual reset in Part 14.10). Without the variable the API still boots, but refuses to let anyone enable 2FA. It is included in the encrypted server-config backup (Part 12.4).*

2.7 Add Swap Space (Critical for 2GB RAM)

The build process uses significant memory. Without swap, it will fail on a 2GB server.

```
# Create a 2GB swap file
fallocate -l 2G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile

# Make it permanent (survives reboot)
echo '/swapfile swap swap defaults 0 0' >> /etc/fstab

# Verify
free -h
# You should see ~2.0G under Swap
```

Part 3: Application Deployment (first install)

This part is for a **brand-new server**. Day-to-day releases do not use it — they go through CI and `deploy.sh` (Part 11). Rebuilding a server from the off-site backup is Part 12.6.

3.1 Clone the Repository (the `prod` branch)

Production always runs the `prod` branch — `deploy.sh` does `git checkout prod && git reset --hard origin/prod`.

```
mkdir -p /var/www/naro-fashion
cd /var/www/naro-fashion
git clone -b prod https://github.com/rjurio/naro-fashion.git .
# Note the dot (.) at the end — clones into current directory, not a subdirectory
```

If the repo is private, you need to authenticate:

```
# Option A: HTTPS with a fine-grained, read-only personal access token
git clone -b prod https://<YOUR_GITHUB_TOKEN>@github.com/rjurio/naro-fashion.git .

# Option B: SSH (add the server's public key to GitHub as a read-only deploy key)
cat ~/.ssh/id_ed25519.pub
git clone -b prod git@github.com:rjurio/naro-fashion.git .
```

> ***Never copy ad-hoc files into*** `/var/www/naro-fashion`***.*** *Untracked files at paths that are later committed make the next deploy's git step fail (Part 14.8). Put one-off helper scripts in* `/tmp/` *instead.*

3.2 Create Environment Files

`.env` files are git-ignored and never touched by `git reset --hard` (which only rewrites tracked files), so they survive every deploy. There are **five** of them:

File	Read by
`/var/www/naro-fashion/.env`	API (`@nestjs/config`), scripts
`/var/www/naro-fashion/apps/api/.env`	API (`@nestjs/config`)
`/var/www/naro-fashion/packages/database/.env`	Prisma (CLI + client — **loaded first**)
`/var/www/naro-fashion/apps/storefront/.env.local`	Storefront build + `deploy.sh` health check (`STOREFRONT_DEFAULT_HOSTS`)
`/var/www/naro-fashion/apps/admin/.env.local`	Admin build

> ***Env file sprawl — the dotenv precedence trap.*** *Three `.env` files are loaded by the API at boot and they do NOT merge the way you expect. Prisma's internal `dotenv` call runs FIRST (when `PrismaClient` is imported, reading `packages/database/.env`), and `dotenv` never overrides a variable that is already set. So **any key that `packages/database/.env` sets wins**, and the later `@nestjs/config` load is a no-op for that key. This burned a CORS fix: `STOREFRONT_URL` was correct in two of the three files but stale in `packages/database/.env`, so `https://www.narofashion.co.tz` got no `Access-Control-Allow-Origin` header. The same trap applies to JWT secrets: an empty `JWT_SECRET=` in `packages/database/.env` silently wins even when the other two files are right.*
>
> ***Rule:*** *keep the three backend `.env` files identical (simplest: write the root one, then copy it to the other two, and repeat the copy after every edit). PM2's `ecosystem.config.js` only sets `NODE_ENV`, `PORT` and `GIT_SHA` — it does not provide application variables.*

Root `.env` (copied to `apps/api/.env` and `packages/database/.env`)

```
cat > /var/www/naro-fashion/.env << 'EOF'
NODE_ENV=production

DATABASE_URL="postgresql://naro_admin:<db_password>@localhost:5432/naro_fashion?
schema=public"

# Required in production: >= 32 chars each, and different (openssl rand -hex 48)
JWT_SECRET="<jwt_secret>"
JWT_REFRESH_SECRET="<jwt_refresh_secret>"
# Defaults; each tenant can override in Admin → Settings → Security → Session Timing
JWT_ACCESS_EXPIRES="15m"
JWT_REFRESH_EXPIRES="7d"

# Required to let admins enable 2FA. NEVER rotate after enrolment.
TWO_FA_ENCRYPTION_KEY="<two_fa_key>"

# CORS + email links. STOREFRONT_URL accepts a comma-separated list.
STOREFRONT_URL="https://narofashion.co.tz,https://www.narofashion.co.tz"
ADMIN_URL="https://admin.narofashion.co.tz"
```

```
API_PUBLIC_URL="https://api.narofashion.co.tz"
NEXT_PUBLIC_API_URL="https://api.narofashion.co.tz/api/v1"

# Default tenant (Instagram env fallback, email branding fallback)
DEFAULT_TENANT_SLUG="naro-fashion"

# Runtime tenant-scope guard: warn (default) | strict | off
TENANT_SCOPE_ENFORCEMENT="warn"

SMTP_HOST=""
SMTP_PORT="587"
SMTP_USER=""
SMTP_PASS=""
SMTP_FROM="noreply@narofashion.co.tz"
EOF
chmod 600 /var/www/naro-fashion/.env
```

Replace every `<placeholder>` with the values from Part 2.5 / 2.6, and the domains with yours. SMTP fields stay empty until Part 9. The complete list of variables (payments, Instagram, AI assistant, retention, order/rental TTLs, etc.) is in **Appendix A**.

```
cd /var/www/naro-fashion
cp .env apps/api/.env
cp .env packages/database/.env
chmod 600 apps/api/.env packages/database/.env

# Sanity check — must print the same 3 lines of length >= 32, and access != refresh
for f in .env apps/api/.env packages/database/.env; do
  awk -F= -v f="$f" '/^JWT_(REFRESH_)?SECRET=/{gsub(/"/,"",$2); print f, $1, length($2)}'
"$f"
done
```

Storefront `.env.local`

```
cat > /var/www/naro-fashion/apps/storefront/.env.local << 'EOF'
NEXT_PUBLIC_API_URL=https://api.narofashion.co.tz/api/v1
NEXT_PUBLIC_TENANT_SLUG=naro-fashion
# Hosts the middleware accepts in production (unknown hosts get 404).
# The FIRST entry is also the Host header deploy.sh uses for its health check.
STOREFRONT_DEFAULT_HOSTS=narofashion.co.tz,www.narofashion.co.tz
EOF
```

> ***Why*** `STOREFRONT_DEFAULT_HOSTS` ***?*** *In production the storefront middleware returns 404 for any Host it cannot resolve to a tenant — including* `127.0.0.1`*. The primary tenant normally resolves via its* `domain` *column (Part 10); this list is a safety net, and* `deploy.sh` *reads its first entry so the post-deploy health check sends a real tenant Host.*
>
> `NEXT_PUBLIC_*` *values are* ***baked in at build time*** *(including into the CSP* `connect-src`*). Changing one requires a rebuild (a deploy), not just a restart.*

Admin `.env.local`

```
cat > /var/www/naro-fashion/apps/admin/.env.local << 'EOF'
NEXT_PUBLIC_API_URL=https://api.narofashion.co.tz/api/v1
EOF
```

3.3 Install Dependencies

```
cd /var/www/naro-fashion
pnpm install --frozen-lockfile
```

This takes 2-5 minutes. `--frozen-lockfile` installs exactly what `pnpm-lock.yaml` says and fails rather than rewriting it. **Never run a plain `pnpm install` (or `pnpm add`) on the VPS** — it can modify the tracked lockfile, and the next deploy then aborts (Part 14.8). Dependency changes are made locally, committed, and deployed.

> *`multer` used to need a manual `pnpm add` on the server. It is now declared in `apps/api/package.json`; that step is obsolete.*

3.4 Set Up the Database Schema (migrations)

Since October 2026 the schema is managed **only** by committed Prisma migrations in `packages/database/prisma/migrations/` (`0_init` baseline + later migrations). `prisma db push --accept-data-loss` is no longer used anywhere in production.

Always use the **local** Prisma CLI (v6.x) via `pnpm exec prisma` from `packages/database/` — never a globally installed `prisma` (v7 has breaking changes) and never `npx prisma` (may fetch a different version).

```
cd /var/www/naro-fashion/packages/database

# Generate the Prisma client
pnpm exec prisma generate

# Fresh, EMPTY database: apply all committed migrations (creates every table
# and the _prisma_migrations bookkeeping table)
pnpm exec prisma migrate deploy

# Should report "Database schema is up to date!"
pnpm exec prisma migrate status

cd /var/www/naro-fashion
```

> ***Why run `migrate deploy` by hand on a fresh install?*** *`deploy.sh` treats a database without a `_prisma_migrations` table as a legacy `db push` database and runs a one-time **baseline check** (Part 11.3, step 5a). An empty database differs from `0_init`, so that check reports drift and aborts the deploy. Running `migrate deploy` once by hand creates the table and the baseline check is skipped from then on.*

Table ownership. All tables must be owned by `naro_admin` (the connection user) — migrations run `ALTER TABLE`, which needs ownership, not just `GRANT ALL`. Tables created by `naro_admin` (as above) are owned correctly. If tables were ever created by `postgres` (e.g. a restore run as the superuser), fix it with the SQL in Part 14.9.

3.5 Run Migration and Seed Scripts

```
cd /var/www/naro-fashion

# Create the first tenant, platform admin, subscription plans, and modules
node packages/database/prisma/migrate-to-multi-tenant.js

# Seed tenant-specific data (site settings, CMS pages, payment methods)
node packages/database/prisma/seed-tenant.js
```

What these scripts create:

Script	Creates
`migrate-to-multi-tenant.js`	Default tenant ("Naro Fashion", slug `naro-fashion`), platform admin (`platform@naro.co.tz`), tenant admin (`admin@narofashion.co.tz`), 3 subscription plans (Starter/Business/Enterprise), all module definitions
`seed-tenant.js`	Site settings, default CMS pages (About, Contact, FAQ, Terms, Privacy, Size Guide, Shipping, Returns), payment methods (Visa, Mastercard, M-Pesa, Tigo Pesa, Airtel Money, etc.)

The default passwords for the two admin accounts are defined in the seed script (and the project `CLAUDE.md`). They are well known — **change both immediately after first login** (Part 13.2), and enable 2FA for both.

System roles, permissions, expense categories and product sizes are seeded automatically by the API on boot (singleton seeders) — no script needed.

3.6 Install the Backup System (before the first deploy)

`deploy.sh` takes a pre-deploy `pg_dump` using `/root/.pgpass` and **aborts if it cannot**. Install the nightly backup system now — it creates `.pgpass`, the cron entry, and runs a first backup:

```
sudo bash /var/www/naro-fashion/scripts/ops/setup-backups.sh
# (no argument = reads the DB password from packages/database/.env)
```

Off-site backups and alerting are set up in Part 12.4–12.5.

3.7 First Build and Start

On a fresh server there is no live build yet, so build once by hand — **sequentially, one app at a time** (a parallel `pnpm build` gets OOM-killed on 2GB; Part 4.10). These are the same commands `deploy.sh` runs:

```
cd /var/www/naro-fashion

# 1. API
pnpm --filter api build
test -f apps/api/dist/main.js && echo "api OK"

# 2. Storefront, then copy its static files INTO the standalone server folder
pnpm --filter storefront build
cp -a apps/storefront/.next/static
```

```
apps/storefront/.next/standalone/apps/storefront/.next/static
cp -a apps/storefront/public       apps/storefront/.next/standalone/apps/storefront/public

# 3. Admin, same
pnpm --filter admin build
cp -a apps/admin/.next/static apps/admin/.next/standalone/apps/admin/.next/static
cp -a apps/admin/public       apps/admin/.next/standalone/apps/admin/public

# 4. Each Next build must contain these (missing = OOM-killed build)
ls apps/storefront/.next/BUILD_ID apps/storefront/.next/routes-manifest.json \
   apps/admin/.next/BUILD_ID apps/admin/.next/routes-manifest.json
```

Then start the three processes (Part 5) and finish Parts 6–10 (Nginx, DNS, SSL, email, tenant domain). Once the site answers on its real domain, run `bash deploy.sh` once by hand (Part 15.2) to prove the full pipeline — including the health checks and the `.deploy/` state files — works on this server before relying on CI.

Part 4: Build Troubleshooting and Production Incidents

This section documents every build error encountered during real deployments and the exact fix for each one (4.1–4.9), followed by the production incidents of 2026 and what the pipeline now does to prevent them (4.10–4.17). Read 4.10–4.17 even if your build succeeds — they explain why `deploy.sh` looks the way it does.

4.1 "Module not found: @/types/model-viewer"

Error message:

```
Module not found: Can't resolve '@/types/model-viewer'
```

Root cause: The `model-viewer.d.ts` type declaration files must exist in both Next.js apps. These files tell TypeScript about the `<model-viewer>` web component (Google's 3D viewer). If they are missing, the build fails because TypeScript does not know what `<model-viewer>` is.

Fix: Create the type declaration files. They must use `declare module 'react'` (not `declare namespace JSX`) because React 19 changed how JSX type augmentation works.

Create `apps/admin/types/model-viewer.d.ts`:

```
/* eslint-disable @typescript-eslint/no-empty-object-type */
import type React from 'react';

declare module 'react' {
  namespace JSX {
    interface IntrinsicElements {
      'model-viewer': React.DetailedHTMLProps<
        React.HTMLAttributes<HTMLElement> & {
          src?: string;
          alt?: string;
          poster?: string;
          'camera-controls'?: boolean | string;
          'auto-rotate'?: boolean | string;
          ar?: boolean | string;
```

```
          'ar-modes'?: string;
          loading?: 'auto' | 'lazy' | 'eager';
          'shadow-intensity'?: string;
        },
        HTMLElement
      >;
    }
  }
}

export {};
```

Create `apps/storefront/types/model-viewer.d.ts`:

```
import "react";

declare module "react" {
  namespace JSX {
    interface IntrinsicElements {
      "model-viewer": React.DetailedHTMLProps<
        React.HTMLAttributes<HTMLElement> & {
          src?: string;
          alt?: string;
          poster?: string;
          "camera-controls"?: boolean | string;
          "auto-rotate"?: boolean | string;
          ar?: boolean | string;
          "ar-modes"?: string;
          loading?: "auto" | "lazy" | "eager";
          style?: React.CSSProperties;
          "shadow-intensity"?: string;
          "environment-image"?: string;
          exposure?: string;
        },
        HTMLElement
      >;
    }
  }
}
```

> ***Why `declare module 'react'` instead of `declare namespace JSX`?*** *React 19 moved the JSX namespace inside the React module. If you use the old `declare namespace JSX` pattern, TypeScript ignores your augmentation silently and the build still fails with "Property 'model-viewer' does not exist on type 'JSX.IntrinsicElements'".*

4.2 "use client directive must be placed before other expressions"

Error message:

```
Error: "use client" directive must be placed before other expressions.
Move the directive to the top of the file to resolve this issue.
```

Root cause: In a file that has `"use client"` at the top (a React Client Component), you also added `export const dynamic = "force-dynamic"` — but you placed the `export` *before* the `"use client"` directive.

Fix: The `"use client"` directive must ALWAYS be the very first line of the file. If you need `export const dynamic`, put it after:

```
// WRONG:
export const dynamic = "force-dynamic";
"use client";

// CORRECT:
"use client";
export const dynamic = "force-dynamic";
```

4.3 "Error occurred prerendering page /auth/login"

Error message:

```
Error occurred prerendering page "/auth/login"
Error: localStorage is not defined
```

or

```
Error occurred prerendering page "/dashboard"
Error: window is not defined
```

Root cause: Next.js tries to prerender (generate static HTML for) all pages at build time. Pages that use `localStorage`, `window`, `document`, or other browser-only APIs crash during this prerender because they run on the server (Node.js) where these APIs do not exist.

Fix: Add `export const dynamic = "force-dynamic"` to the root `app/layout.tsx` of BOTH Next.js apps. This tells Next.js to never prerender any page — all pages are rendered on-demand at request time.

Both layout files should start like this:

```
export const dynamic = "force-dynamic";
import type { Metadata } from "next";
// ... rest of imports
```

This line is already present in the codebase. If it is missing (e.g., after a merge conflict), add it back as the very first line of:

- `apps/storefront/app/layout.tsx`
- `apps/admin/app/layout.tsx`

4.4 "Module not found: Can't resolve 'multer'" (historical)

> ***Resolved in the codebase:*** *`multer` (v2) and `@types/multer` are now declared in `apps/api/package.json`, so a normal `pnpm install --frozen-lockfile` installs them. Do* ***not*** *run `pnpm add` on the VPS (it rewrites the tracked lockfile — Part 14.8); if a dependency is ever missing, add it locally, commit the lockfile, and deploy. The original write-up is kept below for diagnosis.*

Error message (at runtime, not build time):

```
Error: Cannot find module 'multer'
```

The API starts but immediately crashes. PM2 shows the app restarting repeatedly (restart count climbs into hundreds or thousands).

Root cause: pnpm uses strict dependency hoisting. The `multer` package (used for file uploads) must be explicitly listed in `apps/api/package.json`. Unlike npm, pnpm does not let packages access dependencies they did not explicitly declare.

Fix (today): confirm `multer` is in `apps/api/package.json`, then redeploy (which runs `pnpm install --frozen-lockfile`). If it is genuinely missing, add it **on your development machine** (`pnpm add multer @types/multer --filter api`), commit `apps/api/package.json` + `pnpm-lock.yaml`, and push to `prod`.

4.5 ReactQuill ref prop error

Error message:

```
Warning: Function components cannot be given refs.
Attempts to access this ref will fail.
```

or a build error about `ref` on a dynamically imported component.

Root cause: `react-quill-new` is loaded via `next/dynamic` with `{ ssr: false }`. Dynamic imports wrap the component in a way that does not support React `ref` forwarding.

Fix: Do not use the `ref` prop on the dynamically imported ReactQuill component. If you need to access the editor instance, use a callback or state instead.

4.6 TypeScript "Property X does not exist"

Error message:

```
Type error: Property 'imageUrl' does not exist on type 'Category'.
Type error: Property 'totalSpent' does not exist on type 'User'.
```

Root cause: TypeScript strict mode catches missing interface fields. This happens when the API returns fields that are not declared in the frontend TypeScript interfaces.

Fix (proper): Add the missing field to the TypeScript interface in `packages/shared/` or the local type definition.

Check locally with the exact commands CI runs: `npx tsc --noEmit -p apps/api/tsconfig.json` (and `apps/storefront`, `apps/admin`).

> ***Obsolete workaround:*** *v2.0 suggested temporarily setting `typescript: { ignoreBuildErrors: true }` in `next.config.js`. Don't — the CI `verify` job runs `tsc --noEmit` for all three apps independently of `next.config.js`, so a type error blocks the deploy anyway. Fix the error. The codebase builds with `ignoreBuildErrors: false`.*

4.7 Next.js Standalone Mode — Unstyled or Blank Pages

Symptom: The site loads as raw unstyled HTML (or a blank page / "Application error: a client-side exception has occurred"), and the browser console shows 404s for `/_next/static/...` `.css` and `.js` files.

Root cause: Both Next apps build with `output: 'standalone'`, and PM2 runs the standalone server at `apps/<app>/.next/standalone/apps/<app>/server.js`. A standalone server only serves the `/_next/static`

and `public` files that have been **copied into its own folder** — the build does not do this for you.

Fix: `deploy.sh` copies them after every build. If you ever build by hand, copy them yourself:

```
cd /var/www/naro-fashion
for app in storefront admin; do
  sa="apps/${app}/.next/standalone/apps/${app}"
  rm -rf "${sa}/.next/static" "${sa}/public"
  cp -a "apps/${app}/.next/static" "${sa}/.next/static"
  cp -a "apps/${app}/public"       "${sa}/public"
done
pm2 restart naro-storefront naro-admin
```

See 4.13 for the 7-hour outage this caused when the copy was dropped from the deploy script.

4.8 Prisma EPERM on OneDrive (Windows Development Only)

Error message (Windows only):

```
EPERM: operation not permitted, unlink '...\.prisma\client\...'
```

Root cause: OneDrive file sync locks files that Prisma tries to overwrite during `prisma generate`.

Fix: Retry the command — it usually works on the second attempt. This is a Windows development issue only — it does not happen on the Linux production server.

4.9 Out of Memory During Build

Symptom: Build process is killed by the OS, or you see `ENOMEM` or `Killed` in the output — or, worse, nothing at all (see 4.10).

Root cause: 2GB RAM is tight for a Next.js build. Without swap, the OOM killer terminates the build process.

Fix: Ensure swap is enabled (Part 2.7), build **one app at a time** (never root `pnpm build`), and make sure no other application on the shared VPS is building at the same time:

```
free -h
# If no swap line, create it:
fallocate -l 2G /swapfile && chmod 600 /swapfile && mkswap /swapfile && swapon /swapfile
echo '/swapfile swap swap defaults 0 0' >> /etc/fstab
```

4.10 Incident: Parallel Turbo Build = Silent OOM, Incomplete Build

What happened: Root `pnpm build` fans out storefront + admin + api builds **in parallel** via Turborepo. Peak memory exceeded 2GB; the kernel OOM-killed a Next.js build mid-way, the process exited without a clear error, and `.next/standalone/apps/<app>/.next/` was left incomplete (no `routes-manifest.json`, `middleware-manifest.json`, `BUILD_ID`). The homepage rendered as unstyled HTML.

Diagnosis: `ls apps/<app>/.next/` — `BUILD_ID` and `routes-manifest.json` must exist and be non-empty.

Now prevented by `deploy.sh`: builds run sequentially (`pnpm --filter api build`, then `storefront`, then `admin`), and after each Next build the script checks for `BUILD_ID` + `routes-manifest.json`; a missing file

aborts the deploy and restores the pre-build copy, so PM2 is never pointed at a half-written build. **Never run root `pnpm build` on the VPS.**

4.11 Incident: TypeScript Error → "CI Green, Prod Stale"

What happened: `deploy.sh` runs under `set -e`. A single TypeScript error in any app made the build exit non-zero, which aborted the script **before** the PM2 restart. The old CI only reported that SSH had run — the deploy "went green" while the old code kept serving. Symptom: the API rejected a DTO field added in the latest commit (`shippingFee should not exist`). Examples caught this way: a Prisma `orderBy: { createdAt: 'asc' }` literal inferred as `string` (fix: `'asc' as const`), and `@nestjs/jwt` v11 narrowing `expiresIn` (fix: cast the SiteSetting-derived duration).

Now prevented: the CI `verify` job runs `tsc --noEmit` for api, storefront and admin (plus jest and the audit) **before** the `deploy` job may start, and a failed deploy now exits non-zero so the CI run goes red. Still: when a customer-facing bug appears right after a "successful" deploy, first check `GET /api/v1/health` → `commit` (Part 13.1).

4.12 Incident: Silent Deploys — `deploy.sh` Pulled `master`, CI Triggered on `prod`

What happened (caught 2026-05-10): CI triggered on pushes to `prod`, but `deploy.sh` ran `git pull origin master`. Every deploy since 2026-04-25 "succeeded" in under a minute while deploying nothing — three commits, including a security fix bundle and a whole new API module, never reached the server. Found only because a new endpoint returned 404.

Now prevented: `deploy.sh` runs `git fetch origin prod && git checkout prod && git reset --hard origin/prod` and then **asserts** `HEAD == EXPECTED_SHA` (the commit CI verified); a mismatch aborts. After every deploy, verify **live behaviour**, not just the SHA (Part 13).

4.13 Incident: 2026-10-04 — Unstyled Site for ~7 Hours (static copy dropped, distDir rename)

What happened: During the October 2026 deploy rewrite, two things went wrong together:

1. **The standalone static copy was dropped.** The rewrite assumed (wrongly, from out-of-date documentation) that PM2 ran `next start`. PM2 actually runs the standalone `server.js`, which needs `.next/static` + `public` copied into its folder (4.7). Without the copy every CSS/JS asset 404'd.
2. **Building into a side directory and renaming it to `.next`.** To avoid touching the live build, Next apps were built into `.next-build` and then renamed. But `next build` **bakes its `distDir` name into the output** (`required-server-files.json`, server chunks), so the renamed build served HTML whose every `/_next/static` asset returned 404.

The HTML-only health check (`GET /` → 200) stayed green the whole time, so the deploy reported success while customers saw an unstyled, non-interactive site.

Lessons now encoded in `deploy.sh`:

- Next apps build **in place** into `.next`; the live build is first copied to `.next-prev.pending` (restored if the build fails). **Never build Next into another distDir and rename it.**
- After every build, `.next/static` + `public` are copied into `.next/standalone/apps/<app>/`; a missing `server.js` aborts.
- The health check fetches each app's page, extracts a real `/_next/static/...css|js` URL from it and **requires that asset to return 200** — an HTML 200 alone is not enough.
- **Verify runtime facts before changing deploy mechanics** — `pm2 jlist` shows the exact script, cwd and exec mode of each process.

4.14 Incident: Graceful Cluster Reload Left a Stale Worker

What happened (also 2026-10-04): after `.next` was swapped/rebuilt, a graceful `pm2 reload` of the admin app (cluster mode) left the old worker serving from the moved directory; admin kept 404-ing every asset until a hard `pm2 restart`.

Now prevented: `deploy.sh` (`reload_pm2` in `scripts/ops/deploy-lib.sh`) uses a graceful `pm2 reload --only naro-api` for the API (its `dist/` is not moved) but a **hard `pm2 restart --only naro-storefront,naro-admin`** for the Next apps, both with `--update-env` and `GIT_SHA`.

4.15 Prisma: `must be owner of table` / `permission denied for table`

What happened: schema changes run `ALTER TABLE`, which requires **table ownership**, not just `GRANT ALL`. On a database bootstrapped with tables owned by `postgres` (superuser) while the app connects as `naro_admin`, the schema step failed mid-deploy and the script aborted before restart.

Fix (one-time per database, also after a restore run as a different user): run the ownership SQL in Part 14.9. Tables created later by migrations running as `naro_admin` are owned by it automatically.

4.16 CI: A Critical Dependency Advisory Blocks Deploys

What happens: the `verify` job ends with `pnpm audit --prod --audit-level=critical`. When a new **critical** advisory is published for a production dependency, the job fails and **nothing deploys** — even if your change is unrelated. Example: on 2026-10-18 a critical advisory against `handlebars` (used for email templates) blocked deploys until `apps/api` was bumped to `handlebars@^4.7.10`.

Fix: reproduce locally with `pnpm audit --prod --audit-level=critical`, bump the dependency (or add a `pnpm.overrides` entry for a vulnerable transitive dependency), commit the updated `pnpm-lock.yaml`, push. Do not weaken the audit level to get a deploy through, and **never** fix it by running `pnpm add` on the VPS.

4.17 Local Windows Build Fails at "Collecting build traces" (`EPERM symlink`)

Symptom (Windows dev machine only): `next build` fails late with `EPERM: operation not permitted, symlink ...` while "Collecting build traces".

Cause: the standalone output copies traced files using symlinks, which need Developer Mode / admin rights on Windows. Compile, type-check and page generation have already passed by then — it is **not** a code error and does not occur on the Linux VPS. For a local type check use `npx tsc --noEmit -p apps/<app>/tsconfig.json` (the same command CI runs).

Part 5: PM2 Configuration

PM2 is the process manager that keeps the three Naro applications running, restarts them on crash, and starts them on server boot. Remember that the same PM2 daemon also runs **other applications** on this VPS — every command below names the Naro processes explicitly.

5.1 The PM2 Configuration (committed — do not hand-write it)

`ecosystem.config.js` is committed in the repo root and matches the live processes. Do **not** overwrite it on the server (a modified tracked file blocks the next deploy — Part 14.8). Its essentials:

App	`cwd`	`script`	Mode	Env

`naro-api`	`./apps/api`	`dist/main.js`	cluster, 1 instance	`NODE_ENV=productic` `PORT=4000`, `GIT_SHA`
`naro-storefront`	`./apps/storefront`	`.next/standalone/apps/storefront/server.js`	cluster, 1 instance	`NODE_ENV=productic` `PORT=3000`
`naro-admin`	`./apps/admin`	`.next/standalone/apps/admin/server.js`	cluster, 1 instance	`NODE_ENV=productic` `PORT=3001`

Key details:

Setting	Explanation
Standalone `server.js`	The Next apps run the **standalone** server, not `next start`. It only serves the `.next/static` + `public` files copied into `.next/standalone/apps/<app>/` (Part 4.7). Verify what is really running with `pm2 jlist` before changing anything about deploys (Part 4.13)
`GIT_SHA`	Exported by `deploy.sh` and passed through `env` so `GET /api/v1/health` reports the running commit
cluster, 1 instance	Must match the live processes. Changing `exec_mode`/`instances` needs `pm2 delete <name>` + start — a reload will not apply it
No app env vars	The ecosystem only sets `NODE_ENV`, `PORT`, `GIT_SHA`. Everything else comes from the `.env` files (Part 3.2)

> ***Zero downtime is not possible on this box.*** *One instance per app on 2GB RAM means a reload/restart is a stop/start: a few seconds of 502s per app. The deploy minimises the window (all builds finish before anything restarts) but cannot remove it.*

5.2 Copy Static Files

Covered by `deploy.sh` on every deploy, and by Part 3.7 on a first install. If you ever build by hand, use the loop in Part 4.7 **before** restarting the Next apps.

5.3 Start the Applications (first install only)

```
cd /var/www/naro-fashion

# Start the three Naro apps (never a bare `pm2 start` of everything)
GIT_SHA=$(git rev-parse HEAD) pm2 start ecosystem.config.js --only naro-api,naro-
storefront,naro-admin

# Save the process list (so PM2 restores it on reboot)
pm2 save

# Enable PM2 to start on boot (once per server)
pm2 startup
# This prints a command like:
```

```
#   sudo env PATH=$PATH:/usr/bin pm2 startup systemd -u root --hp /root
# Copy and run that exact command!
```

After this, `deploy.sh` manages the processes; you should not need `pm2 start` again.

5.4 Verify the Apps Are Running

```
pm2 list
```

You should see the three `naro-*` processes `online` (alongside any other applications on the server):

```
┌────┬────────────────────┬──────────┬────────┬──────────┬──────────┬──────────┐
│ id │ name               │ mode     │ status │ cpu      │ mem      │ restarts │
├────┼────────────────────┼──────────┼────────┼──────────┼──────────┼──────────┤
│ 0  │ naro-api           │ cluster  │ online │ 0.5%     │ 150 MB   │ 0        │
│ 1  │ naro-storefront    │ cluster  │ online │ 0.3%     │ 90 MB    │ 0        │
│ 2  │ naro-admin         │ cluster  │ online │ 0.2%     │ 80 MB    │ 0        │
│ …  │ (other apps)       │ …        │ …      │ …        │ …        │ …        │
└────┴────────────────────┴──────────┴────────┴──────────┴──────────┴──────────┘
```

If any shows `errored` or has a climbing restart count, check logs:

```
pm2 logs naro-api --lines 50 --nostream
pm2 logs naro-storefront --lines 50 --nostream
pm2 logs naro-admin --lines 50 --nostream
```

To see exactly what PM2 runs (script path, cwd, exec mode, env):

```
pm2 jlist | node -e 'let s="";process.stdin.on("data",d=>s+=d).on("end",()=>{for(const p of
JSON.parse(s)){const e=p.pm2_env;if(p.name.startsWith("naro-
"))console.log(p.name,e.exec_mode,e.pm_cwd,e.pm_exec_path,e.status,"GIT_SHA="+
(e.GIT_SHA||""))}})'
```

5.5 Common PM2 Commands

```
pm2 list                                # Status of all processes (Naro + others)
pm2 logs naro-api                       # Stream API logs (Ctrl+C to stop)
pm2 logs naro-api --lines 100 --nostream   # Last 100 lines, no streaming
pm2 restart naro-api                    # Restart ONE app
pm2 restart naro-storefront naro-admin     # Restart the two Next apps
pm2 monit                               # Live CPU/memory monitor
pm2 info naro-api                       # Detailed info for one app
pm2 save                                # Save current process list
```

> ***Never*** *`pm2 restart all`, `pm2 stop all`, `pm2 delete all` or `pm2 flush` without a name on this server — they hit the other applications too.*

Part 6: Nginx Configuration

Nginx acts as a reverse proxy — it receives all HTTP/HTTPS traffic and routes it to the correct application based on the domain name.

6.1 Create the Nginx Configuration

```
cat > /etc/nginx/sites-available/narofashion << 'NGINX'
# ====================================
# Storefront — narofashion.co.tz
# ====================================
server {
    listen 80;
    server_name narofashion.co.tz www.narofashion.co.tz;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}

# ====================================
# Admin Dashboard — admin.narofashion.co.tz
# ====================================
server {
    listen 80;
    server_name admin.narofashion.co.tz;

    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}

# ====================================
# API — api.narofashion.co.tz
# ====================================
```

```
server {
    listen 80;
    server_name api.narofashion.co.tz;

    # Allow large file uploads (product images, 3D models up to 25MB)
    client_max_body_size 30M;

    location / {
        proxy_pass http://127.0.0.1:4000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }

    # Cache static uploads (product images, payment icons) for 7 days
    location /uploads/ {
        proxy_pass http://127.0.0.1:4000/uploads/;
        add_header Cache-Control "public, max-age=604800";
    }
}
NGINX
```

Replace `narofashion.co.tz` with your actual domain throughout the file if different.

Key settings explained:

Setting	Why
`proxy_http_version 1.1`	Enables WebSocket support and keep-alive connections
`proxy_set_header Host $host`	Passes the original domain to the app (needed for tenant resolution)
`proxy_set_header X-Real-IP`	Passes the real client IP (not Nginx's IP) to the app
`proxy_set_header X-Forwarded-Proto`	Tells the app whether the request came via HTTP or HTTPS
`client_max_body_size 30M`	API accepts file uploads up to 25MB (3D models). 30M gives buffer for multipart overhead
`/uploads/` with `Cache-Control`	Product images and payment icons are cached by the browser for 7 days

6.2 Enable the Site

```
# Create symlink to enable the site
ln -sf /etc/nginx/sites-available/narofashion /etc/nginx/sites-enabled/
```

```
# Remove the default Nginx welcome page
rm -f /etc/nginx/sites-enabled/default

# Test the configuration for syntax errors
nginx -t
# Expected: nginx: configuration file /etc/nginx/nginx.conf test is successful

# Reload Nginx to apply
systemctl reload nginx
```

6.3 Verify Nginx Is Working

At this point (before SSL), you can test with the server IP:

```
# From the server itself:
# Storefront: in production the middleware 404s unknown hosts (127.0.0.1 included),
# so send a real tenant Host header
curl -s -H "Host: narofashion.co.tz" http://127.0.0.1:3000 | head -5
curl -s http://127.0.0.1:3001/login | head -5   # Admin
curl -s http://127.0.0.1:4000/api/v1/health      # API:
{"status":"ok","db":"ok","commit":...}
```

If any of these return nothing or errors, check `pm2 list` and `pm2 logs naro-<app> --nostream`.

> ***Shared server:*** *the Nginx on this VPS also serves other sites (`jumuiya`, `kiuruwi`, `sportflow`). Always run `nginx -t` before `systemctl reload nginx` — a syntax error in any site file blocks the reload for all of them. Nginx config is included in the nightly encrypted server-config backup (Part 12.4).*

Part 7: DNS Setup

DNS tells the internet where to find your domain. You need 4 A records pointing to your VPS IP.

7.1 DNS Records Needed

Type	Name/Host	Value/Points to	TTL
A	`@` (root domain)	`YOUR_VPS_IP` (e.g., `80.240.30.107`)	3600
A	`www`	`YOUR_VPS_IP`	3600
A	`admin`	`YOUR_VPS_IP`	3600
A	`api`	`YOUR_VPS_IP`	3600

7.2 Option A: Use Vultr DNS (Recommended)

If your domain registrar does not have a DNS zone editor (common with Tanzanian registrars like Habari Node), point your nameservers to Vultr and manage DNS there.

Step 1: Add domain to Vultr DNS

1. Log in to **https://my.vultr.com**
2. Go to **Products** > **Network** > **DNS**

3. Click **Add Domain**
4. Enter your domain: `narofashion.co.tz`
5. Select your VPS as the default IP

Step 2: Add A records

In Vultr DNS for your domain, add:

- `@` → `YOUR_VPS_IP` (A record)
- `www` → `YOUR_VPS_IP` (A record)
- `admin` → `YOUR_VPS_IP` (A record)
- `api` → `YOUR_VPS_IP` (A record)

Optionally add a wildcard for future tenant domains:

- `*` → `narofashion.co.tz` (CNAME record)

Step 3: Point nameservers at your registrar

Log in to your domain registrar (e.g., Habari Node at `hosting.habari.co.tz`) and change the nameservers to:

- `ns1.vultr.com`
- `ns2.vultr.com`

> ***Habari Node specific:*** *If you cannot find the nameserver settings, contact their support:*
>
> - *Portal: https://hosting.habari.co.tz/ (open a support ticket)*
> - *Email: software@habari.co.tz*
> - *Phone: +255 659 074 444*
>
> *Tell them: "I need to change nameservers for narofashion.co.tz to ns1.vultr.com and ns2.vultr.com."*

7.3 Option B: Use Cloudflare (Free Alternative)

Cloudflare provides free DNS with additional features like DDoS protection and CDN.

1. Go to **https://dash.cloudflare.com** and create a free account
2. Click **Add Site** and enter your domain
3. Select the **Free** plan
4. Add the 4 A records listed in Section 7.1
5. Cloudflare will give you 2 nameservers (e.g., `anna.ns.cloudflare.com`, `bob.ns.cloudflare.com`)
6. Go to your registrar and change nameservers to the Cloudflare nameservers

> ***Note:*** *If using Cloudflare, make sure the proxy (orange cloud) is set to "DNS only" (grey cloud) for the API subdomain, otherwise WebSocket connections and file uploads may have issues.*

7.4 Option C: Direct DNS at Registrar

If your registrar provides a DNS zone editor:

1. Log in to your registrar's control panel
2. Find the DNS management / Zone Editor
3. Add the 4 A records from Section 7.1
4. Delete any existing A records that point elsewhere

7.5 Verify DNS Propagation

After making DNS changes, wait 5-30 minutes for A record changes, or up to 24-48 hours for nameserver changes.

```
# Test from any terminal (or use the server):
dig narofashion.co.tz +short
# Expected: 80.240.30.107

dig admin.narofashion.co.tz +short
# Expected: 80.240.30.107

dig api.narofashion.co.tz +short
# Expected: 80.240.30.107
```

If `dig` is not available, use `nslookup`:

```
nslookup narofashion.co.tz
```

Or check online at **https://dnschecker.org** to see global propagation.

DO NOT proceed to SSL setup (Part 8) until DNS is fully propagated. Certbot will fail if the domain does not resolve to your server IP.

Part 8: SSL Certificates

Let's Encrypt provides free SSL certificates. Certbot automates the entire process.

8.1 Install SSL Certificates

```
certbot --nginx \
  -d narofashion.co.tz \
  -d www.narofashion.co.tz \
  -d admin.narofashion.co.tz \
  -d api.narofashion.co.tz \
  --email your-email@example.com \
  --agree-tos \
  --non-interactive
```

Replace:

- Domain names with your actual domain
- Email with your actual email (Let's Encrypt sends expiry reminders here)

On success, Certbot will:

1. Obtain certificates for all 4 domains
2. Automatically modify the Nginx config to add SSL listeners (port 443)
3. Add HTTP-to-HTTPS redirects (port 80 redirects to 443)

8.2 Verify Auto-Renewal

```
certbot renew --dry-run
```

This simulates a renewal. If it says "Congratulations, all simulated renewals succeeded", auto-renewal is working. Certbot installs a systemd timer that runs twice daily and renews certificates before they expire (every 90 days).

8.3 Troubleshooting SSL

"Challenge failed for domain..."

This means Certbot could not verify you own the domain. Causes:

- DNS not propagated yet (most common). Wait and retry.
- Wrong A record (points to wrong IP). Verify with `dig domain +short`.
- Firewall blocking port 80. Check: `ufw status` should show `Nginx Full` allowed.

Certificates for fewer domains than requested:

If one domain fails, Certbot may succeed for the others. Run Certbot again with just the failed domain:

```
certbot --nginx -d admin.narofashion.co.tz --email your@email.com --agree-tos
```

View installed certificates:

```
certbot certificates
```

Part 9: Email Setup (Brevo)

Brevo (formerly Sendinblue) provides free transactional email (300 emails/day).

9.1 Create Brevo Account

1. Go to **https://www.brevo.com** and sign up (Google OAuth works)
2. Choose the **Free plan** — scroll down past the paid plans; the free option is at the bottom
3. Complete the setup wizard

9.2 Generate SMTP Key

1. In the Brevo dashboard, click the **gear icon** (Settings) in the bottom-left
2. Navigate to **SMTP & API** > **SMTP**
3. Click **"Generate a new SMTP key"**
4. Name it `naro-fashion-prod`, select **Standard** variant
5. Click **Generate** and **copy the key immediately** (it will not be shown again)

You will have:

```
SMTP Server:  smtp-relay.brevo.com
Port:         587
Login:        your-account-id@smtp-brevo.com  (shown on the SMTP page)
Password:     xsmtpsib-<long-key>
```

9.3 Add Sender in Brevo

1. Go to **Settings** > **Senders, Domains & Dedicated IPs** > **Senders**
2. Click **Add a sender**
3. Enter:
    - **From Name:** `Naro Fashion`

- **From Email:** `noreply@yourdomain.com` (e.g., `noreply@narofashion.co.tz`)

4. Click **Add**
5. Choose **"Authenticate the domain yourself"** > **Continue**

9.4 Authenticate Domain (DNS Records)

Brevo will show you DNS records to add. Go to your DNS provider (Vultr DNS, Cloudflare, or your registrar) and add:

Record 1 — Brevo verification code:

Type	Name	Value	TTL
TXT	`@` (or blank)	`brevo-code:<your-verification-code>`	3600

Record 2 — DKIM signature 1:

Type	Name	Value	TTL
CNAME	`brevo1._domainkey`	`b1.yourdomain-tld.dkim.brevo.com`	3600

Record 3 — DKIM signature 2:

Type	Name	Value	TTL
CNAME	`brevo2._domainkey`	`b2.yourdomain-tld.dkim.brevo.com`	3600

> ***Note:*** *For* `.co.tz` *domains, the DKIM value uses hyphens: e.g.,* `b1.narofashion-co-tz.dkim.brevo.com`

Record 4 — DMARC policy:

Type	Name	Value	TTL
TXT	`_dmarc`	`v=DMARC1; p=none; rua=mailto:rua@dmarc.brevo.com`	3600

9.5 Verify Domain in Brevo

1. Go back to Brevo and click **Authenticate** / **Check Configuration**
2. Wait a few minutes for DNS to propagate
3. Once verified, your domain shows a green checkmark
4. Emails from `noreply@yourdomain.com` will now pass SPF, DKIM, and DMARC checks (delivered to inbox, not spam)

9.6 Update Environment Variables on VPS

```
ssh -o StrictHostKeyChecking=yes root@YOUR_VPS_IP

cd /var/www/naro-fashion
# Update these lines in ALL THREE backend env files (Part 3.2 — the
# packages/database/.env value wins if they disagree):
#   .env   apps/api/.env   packages/database/.env
# SMTP_HOST="smtp-relay.brevo.com"
# SMTP_PORT="587"
# SMTP_USER="your-account-id@smtp-brevo.com"
```

```
# SMTP_PASS="<smtp_key>"
# SMTP_FROM="noreply@yourdomain.com"
nano .env; nano apps/api/.env; nano packages/database/.env
grep -H '^SMTP_HOST=' .env apps/api/.env packages/database/.env   # must agree
```

Then restart only the API:

```
pm2 restart naro-api --update-env
```

9.7 What Emails the Platform Sends

Email Type	Trigger	Template
Order confirmation	Customer places an order	`order-confirmation.hbs`
Password reset	User clicks "Forgot Password"	`password-reset.hbs`
Rental pickup reminder	Daily cron (8:00 AM) for upcoming pickups	`rental-reminder.hbs`
Overdue rental alert	Daily cron (9:00 AM) for overdue returns	`overdue-rental.hbs`
Admin prep reminder	Item needs preparation before pickup	`admin-prep-reminder.hbs`
ID verification status	Admin approves/rejects National ID	`id-verification.hbs`
Newsletter campaign	Admin sends via newsletter module	`newsletter.hbs`
Contact acknowledgement	Customer submits contact form	`contact-acknowledgement.hbs`
Contact reply	Admin replies to contact submission	`contact-reply.hbs`

Part 10: Tenant Configuration

The storefront resolves which tenant to show based on the domain. You must set the tenant's domain in the database.

10.1 Set Tenant Domain

```
# Connect to the database and update the tenant domain
sudo -u postgres psql -d naro_fashion -c "UPDATE \"Tenant\" SET \"domain\" =
'yourdomain.com' WHERE \"slug\" = 'naro-fashion';"
```

Replace `yourdomain.com` with your actual domain (e.g., `narofashion.co.tz`). Do NOT include `https://` or `www.`.

```
# Restart the storefront so middleware picks up the domain
pm2 restart naro-storefront
```

10.2 Verify Tenant Resolution

```
# Test that the storefront can resolve the tenant
curl -s https://yourdomain.com | head -20
# Should return HTML (not a "Store not found" error)
```

10.3 Why Not Use a Node Script?

You might think of running a Prisma script to update the tenant. **Do not use `node -e` scripts with Prisma on the server from arbitrary directories** (like `/tmp`). pnpm's node_modules resolution is directory-dependent, and `@prisma/client` will not be found outside the project directory. Using `psql` directly is the most reliable method.

Part 11: CI/CD Pipeline and deploy.sh

Production is deployed by pushing to the `prod` branch. GitHub Actions verifies the commit, then SSHes to the VPS and runs `deploy.sh`. This part describes that pipeline exactly as it is in `.github/workflows/deploy-prod.yml`, `deploy.sh` and `scripts/ops/deploy-lib.sh`.

11.1 How It Works

```
git push origin prod
        │
        ▼
GitHub Actions: "Verify & Deploy to Production"
        │
        ├── job `verify`  (also runs on pull requests)
        │     shell checks (bash -n + ShellCheck) on deploy.sh, scripts/ops/*.sh
        │     pnpm install --frozen-lockfile
        │     prisma generate
        │     migrations-vs-schema drift check (throwaway Postgres 16 service)
        │     tsc --noEmit: api, storefront, admin
        │     API jest
        │     pnpm audit --prod --audit-level=critical
        │
        ▼ (only if verify is green, only for pushes to prod)
        └── job `deploy`  (concurrency group "deploy-prod": never two at once)
              SSH (pinned ECDSA host-key fingerprint) → /var/www/naro-fashion
              git fetch/checkout/reset to origin/prod (refreshes deploy.sh itself)
              EXPECTED_SHA=<github.sha> bash deploy.sh
                    │
                    ▼
              deploy.sh: reset + SHA assert → frozen install → prisma generate
                 → PRE-DEPLOY pg_dump → migrate deploy (+ baseline/drift guards)
                 → sequential in-place builds (api → storefront → admin)
                 → standalone static copy → swap → pm2 reload/restart
                 → health checks (incl. a real /_next/static asset per app)
                 → on failure: AUTOMATIC ROLLBACK, exit non-zero (CI red)
```

11.2 Set Up SSH Key for GitHub Actions

On the VPS:

```
# Generate a deploy key (no passphrase)
ssh-keygen -t ed25519 -f ~/.ssh/deploy_key -N '' -C 'github-actions-deploy'

# Allow this key to SSH into the server
cat ~/.ssh/deploy_key.pub >> ~/.ssh/authorized_keys
```

11.3 Configure GitHub Secrets

Go to your GitHub repository > **Settings** > **Secrets and variables** > **Actions** > **New repository secret** and add:

Secret Name	Value
`VPS_HOST`	Your VPS IP (e.g., `80.240.30.107`)
`VPS_USER`	`root`
`VPS_SSH_KEY`	The **private** key content from the server: run `cat ~/.ssh/deploy_key` on the VPS and paste the entire output including the `-----BEGIN/END-----` lines

Or use the GitHub CLI (if authenticated) — **from a trusted network only** (Part 1.3):

```
echo "80.240.30.107" | gh secret set VPS_HOST
echo "root" | gh secret set VPS_USER
ssh -o StrictHostKeyChecking=yes root@80.240.30.107 "cat ~/.ssh/deploy_key" | gh secret set
VPS_SSH_KEY
gh secret list
```

Host-key pinning. The deploy step passes `fingerprint: SHA256:2AuvcjqqaC2wpZpD8LVeKiWNgmnS7HybLI+M93wUYxs` to `appleboy/ssh-action`, so a man-in-the-middle cannot receive the deploy. It pins the **ECDSA** key because that is the key the action's SSH library (drone-ssh / Go `x/crypto`) negotiates. If the server is ever rebuilt, its host keys change: verify the new ECDSA fingerprint from the Vultr console (Part 1.3) and update the `fingerprint:` line in `.github/workflows/deploy-prod.yml`, or every deploy will fail at the SSH step.

11.4 The GitHub Actions Workflow

File: `.github/workflows/deploy-prod.yml`. The parts an operator needs to know:

Item	Value / behaviour
Triggers	`push` to `prod` (verify + deploy); `pull_request` (verify only)
Concurrency	Pushes share group `deploy-prod` and are **not** cancelled — deploys are serialised. PR runs cancel superseded runs
`verify` → shell checks	`bash -n` and `shellcheck --severity=error` on `deploy.sh` and `scripts/ops/*.sh` (they run as root on the VPS)
`verify` → Node/pnpm	Node from `.nvmrc` (22), pnpm from `package.json packageManager` (pnpm 10.12.1), `pnpm install --frozen-lockfile`

`verify` → migrations check	`prisma migrate diff --from-migrations prisma/migrations --to-schema-datamodel prisma/schema.prisma --shadow-database-url … --exit-code` against a throwaway `postgres:16` service. **Fails if `schema.prisma` was changed without a committed migration**
`verify` → typecheck	`npx tsc --noEmit -p apps/{api,storefront,admin}/tsconfig.json`
`verify` → tests	`pnpm --filter api exec jest --ci` — the suite is fully green, so any failure is a real regression
`verify` → audit	`pnpm audit --prod --audit-level=critical` — a new critical advisory blocks deploys (Part 4.16)
`deploy`	`needs: verify`; only for pushes to `prod`; `appleboy/ssh-action@v1`, `command_timeout: 30m`, passes `EXPECTED_SHA=${{ github.sha }}`

The remote script the `deploy` job runs:

```
set -e
cd /var/www/naro-fashion
# Refresh deploy.sh itself first so this run uses the committed version
# (deploy.sh does the authoritative reset + SHA check).
git fetch origin prod
git checkout -f prod
git reset --hard origin/prod
EXPECTED_SHA="$EXPECTED_SHA" bash deploy.sh
```

11.5 What deploy.sh Does

`deploy.sh` runs under `set -euo pipefail`; every stage aborts the deploy on failure. It sources `scripts/ops/deploy-lib.sh` (health checks, rollback helpers) **after** the git reset, so it always uses the committed helpers.

#	Stage	On failure
1	`git fetch origin prod`, `git checkout prod`, `git reset --hard origin/prod`; **assert `HEAD == EXPECTED_SHA`** (skipped with a warning on a manual run without it); determine the currently live commit from `.deploy/current_sha` → the API's `/health` commit → `ORIG_HEAD`	abort, nothing changed
2	`pnpm install --frozen-lockfile`	abort
3	`prisma generate` (in `packages/database`)	abort
4	**Pre-deploy backup**: `BACKUP_TAG=predeploy-<sha7> bash scripts/ops/pg-backup.sh` → `/var/backups/naro/postgres/naro_fashion-predeploy-<sha7>-<timestamp>.dump`. The path and the exact `pg_restore` command are printed in the log	**abort — schema untouched** (check `/root/.pgpass` and `/var/log/naro-pg-backup.log`)

5a	**One-time baseline** — only if the DB has no `_prisma_migrations` table (a legacy `db push` DB): replays `0_init` into a throwaway shadow DB and diffs it against the live DB. No difference → `prisma migrate resolve --applied 0_init` (no SQL run). Any difference → abort with the SQL printed	abort on drift; reconcile by hand (Part 15.4)
5b	`prisma migrate status` (informational), then **`prisma migrate deploy`** — applies pending committed migrations, forward-only	abort, old build still live
5c	**Drift guard**: `prisma migrate diff --from-schema-datasource … --to-schema-datamodel … --exit-code`. A non-empty diff means `schema.prisma` changed without a migration	abort **before building** — old build still live
6	**Sequential, in-place builds**: copy `apps/api/dist` → `dist-prev.pending`, `pnpm --filter api build` (must produce `dist/main.js`); for storefront then admin: copy `.next` → `.next-prev.pending` (cache excluded), `pnpm --filter <app> build`, verify `BUILD_ID` + `routes-manifest.json`, require `.next/standalone/apps/<app>/server.js`, copy `.next/static` + `public` into the standalone folder	abort; **pre-build copies restored**; processes not reloaded
7	**Swap**: promote `*.prev.pending` → `.next-prev` / `dist-prev`; write `.deploy/previous_sha` + `.deploy/current_sha`; `pm2 reload ecosystem.config.js --only naro-api --update-env` + `pm2 restart ecosystem.config.js --only naro-storefront,naro-admin --update-env` (both with `GIT_SHA`); `pm2 save`	**automatic rollback**
8	**Health checks** (60 s each): API `http://127.0.0.1:4000/api/v1/health` → 200; storefront `http://127.0.0.1:3000/` → 2xx/3xx **with `Host:` = first entry of `STOREFRONT_DEFAULT_HOSTS`** (else `narofashion.co.tz`); admin `http://127.0.0.1:3001/` → 2xx/3xx; **plus one `/_next/static/*.css|js` asset referenced by the storefront home page and by the admin `/login` page must return 200**. Then a soft check that `/health` mentions the deployed SHA	**automatic rollback**

Why each guard exists is in Part 4.10–4.15.

> ***Brief side effects during a deploy.*** *Pages can misbehave for the few minutes a Next app is building in place, and `nest build` rewrites `apps/api/dist` in place — the running API keeps its code in memory, but on-demand reads from `dist` (Handlebars email templates) can fail during the ~1 minute API build. Then a few seconds of 502s per app during the restart (Part 5.1). Deploy outside peak hours (Tanzania evenings are busiest).*

11.6 Automatic Rollback

If anything fails **after the swap** (a reload error, or any health check not green within 60 s), `deploy.sh`:

1. moves `.next-prev` back to `.next` for each Next app and `dist-prev` back to `dist` (the bad builds are kept as `.next-failed` / `dist-failed` for post-mortem);
2. `git reset --hard <previous_sha>`, re-runs `pnpm install --frozen-lockfile` only if the lockfile differs, and `prisma generate` — so the source tree matches the running build;
3. reloads the API / hard-restarts the Next apps with the old `GIT_SHA`;
4. re-runs the health checks and reports whether the rollback is healthy;
5. prints the pre-deploy dump path + `pg_restore` command and **exits non-zero** (CI goes red, GitHub emails the pusher).

The database is never rolled back automatically. Migrations are forward-only and must be backward compatible (expand/contract, Part 15.3), so the previous build runs against the migrated schema. Restore the dump only if it truly cannot (Part 12.5).

If the `*-prev` artifacts are missing (e.g. the first deploy on a new server), automatic rollback is impossible; the script prints PM2 status/logs and the DB restore help and exits — fix forward by hand.

11.7 Deploy State on the VPS (all git-ignored)

Path	Meaning
`apps/<app>/.next`	served Next build
`apps/<app>/.next-prev.pending`	copy of the live build taken before an in-place build (restored on build failure, else promoted to `.next-prev`)
`apps/<app>/.next-prev`	previous build — the rollback target
`apps/<app>/.next-failed`, `apps/api/dist-failed`	build swapped out by a rollback (debugging)
`apps/api/dist`, `dist-prev`, `dist-prev.pending`	served / previous / pre-build-copy API build
`.deploy/current_sha`	commit whose build is live
`.deploy/previous_sha`	commit whose build is in the `*-prev` dirs (absent = nothing to roll back to)
`.deploy/rolled_back_from`	commit the last rollback moved away from

Disk cost: roughly 2× each `.next` — fine on the 64GB disk. Check with `df -h /` occasionally.

11.8 How to Deploy

Workflow: develop on `master`, deploy from `prod`.

```
# 1. Work on master normally
git add <files>
git commit -m "your changes"
git push origin master

# 2. When ready to deploy to production:
git checkout prod
git merge master --no-edit
git push origin prod    # Triggers verify → deploy

# 3. Switch back to your working branch
git checkout master
```

Monitor the deployment:

```
gh run list --workflow "Verify & Deploy to Production" --limit 3
gh run watch <run-id>
gh run view <run-id> --log-failed     # first stop when a run is red
```

Or go to GitHub > your repo > **Actions**. A red `verify` means nothing was deployed. A red `deploy` means either the deploy aborted before going live (old build still serving) or it rolled back automatically — the log says which. In both cases, check Part 13.

11.9 Create the prod Branch (First Time Only)

```
# On your local machine, from the master branch:
git checkout master
git checkout -b prod
git push origin prod

# Switch back to your working branch
git checkout master
```

Part 12: Firewall, Security and Backups

12.1 Configure UFW Firewall

```
# Allow SSH (port 22)
ufw allow OpenSSH

# Allow HTTP and HTTPS (ports 80 and 443)
ufw allow 'Nginx Full'

# Enable the firewall
ufw enable
# Type "y" when prompted

# Verify
ufw status
```

Expected output:

```
Status: active

To                         Action      From
--                         ------      ----
OpenSSH                    ALLOW       Anywhere
Nginx Full                 ALLOW       Anywhere
OpenSSH (v6)               ALLOW       Anywhere (v6)
Nginx Full (v6)            ALLOW       Anywhere (v6)
```

IMPORTANT: Only 3 ports are open: 22 (SSH), 80 (HTTP), 443 (HTTPS). PostgreSQL (5432) and the app ports (3000/3001/4000) are NOT exposed to the internet — they only listen behind Nginx / on localhost.

12.2 Secure SSH

Current state (October 2026): root password login is still enabled on production. Recommended hardening — do it **only after** confirming key-based login works, and **only from a trusted network** (Part 1.3):

```
# 1. In a NEW terminal on a trusted network, confirm key login works without a password:
ssh -o StrictHostKeyChecking=yes -o PasswordAuthentication=no root@80.240.30.107 'echo key-login-ok'

# 2. Then, on the server:
nano /etc/ssh/sshd_config
```

Find and change these lines (also check `/etc/ssh/sshd_config.d/*.conf`, which can override them — cloud images often ship a file there that re-enables passwords):

```
PasswordAuthentication no
PermitRootLogin prohibit-password
```

```
sshd -t && systemctl restart ssh     # Ubuntu 24.04 service name is "ssh"
```

Keep the existing session open until you have confirmed a new key-based login works. The Vultr web console remains a fallback if you lock yourself out.

Host keys and interception. Always connect with `-o StrictHostKeyChecking=yes`. Known fingerprints are in Part 1.3. A host-key warning from the office network is interception, not a server change — disconnect and use another network. Never `ssh-keygen -R` the server.

12.3 Database Backups — Nightly Local Dumps (LIVE since 2026-06-15)

Installed by `scripts/ops/setup-backups.sh` (Part 3.6). Idempotent — safe to re-run.

Item	Value
Script	`/usr/local/bin/naro-pg-backup.sh` (installed copy of `scripts/ops/pg-backup.sh`)
Schedule	`/etc/cron.d/naro-pg-backup` — daily **03:15 UTC** (06:15 EAT)
Output	`/var/backups/naro/postgres/naro_fashion-YYYY-MM-DD_HHMMSSZ.dump`
Format	`pg_dump --format=custom --compress=9 --no-owner --no-acl`
Retention	30 days rolling (pre-deploy dumps included)
Log	`/var/log/naro-pg-backup.log`
Credentials	`/root/.pgpass` (mode 600): `localhost:5432:naro_fashion:naro_admin:<db_password>`
Alerting	pings `HEALTHCHECK_PGBACKUP_URL` (`/start`, success, `/fail`) from `/etc/naro-backup.env`

Pre-deploy dumps: every deploy runs the same script with `BACKUP_TAG=predeploy-<sha7>` (file `naro_fashion-predeploy-<sha7>-<timestamp>.dump`) and aborts if it fails. Tagged runs intentionally do **not** ping Healthchecks, so a deploy can never mask a missed nightly run.

> ***Important:*** *cron runs the installed copy in `/usr/local/bin`, not the repo file. After a commit changes `scripts/ops/pg-backup.sh` or `offsite-backup.sh`, re-install it:*

```
install -m 0755 /var/www/naro-fashion/scripts/ops/pg-backup.sh      /usr/local/bin/naro-pg-backup.sh
install -m 0755 /var/www/naro-fashion/scripts/ops/offsite-backup.sh /usr/local/bin/naro-offsite-backup.sh
```

Daily verification:

```
ls -lh /var/backups/naro/postgres/ | tail -5
tail -30 /var/log/naro-pg-backup.log
ls /var/backups/naro/postgres/naro_fashion-$(date -u +%Y-%m-%d)*.dump
```

A healthy run ends with a line like `OK naro_fashion backups: N files, X total`.

Manual ad-hoc dump (e.g. before running a data script):

```
BACKUP_TAG=manual-$(date -u +%H%M) bash /var/www/naro-fashion/scripts/ops/pg-backup.sh
```

12.4 Off-site Backups — Cloudflare R2 (LIVE since 2026-10-18)

Local dumps share a disk with the database, so they do not survive disk failure, the VPS being destroyed, or ransomware. `scripts/ops/offsite-backup.sh` copies everything needed to rebuild the box to an S3-compatible bucket on **Cloudflare R2** (owner's account, private bucket, free tier).

Item	Value
Script	`/usr/local/bin/naro-offsite-backup.sh` (installed copy of `scripts/ops/offsite-backup.sh`)
Schedule	`/etc/cron.d/naro-offsite-backup` — daily **04:00 UTC**, 45 min after the dump
Tools	`rclone` (upload) + `age` (encryption) — `apt install -y rclone age`
Config	`/etc/naro-backup.env` (root, mode 600): bucket name, R2 endpoint, access key ID + secret, `age` recipient (public key), Healthchecks ping URLs. **The values live only in that file and the owner's password manager — they are deliberately not in this guide**
Log	`/var/log/naro-offsite-backup.log`
Alerting	pings `HEALTHCHECK_BACKUP_URL`

What lands where:

Source	Bucket path	Encrypted
`/var/backups/naro/postgres/*.dump` (nightly + pre-deploy)	`postgres/<name>.dump.age`	yes (`age`)
`apps/api/uploads/` (public product/CMS images)	`uploads/` (mirror)	no — already public via the site
`apps/api/private-uploads/` (national ID documents)	`private-uploads/private-uploads-<ts>.tar.gz.age`	**always** — the script refuses to

		upload without a key
Server config not in git: the three backend `.env` files, `apps/storefront/.env*`, `apps/admin/.env*`, `/etc/nginx`, `/etc/letsencrypt`, `/etc/cron.d/naro-pg-backup`, `/etc/cron.d/naro-offsite-backup`	`server-config/server-config-<ts>.tar.gz.age` (one per night)	**always** — refuses without a key

Encryption model. `age` public-key encryption: the server holds only the *public* recipient, so it can encrypt but **cannot decrypt**. The **private key is held off-server by the owner** (password manager + an offline copy). A VPS compromise therefore does not expose old backups — but **without the private key nothing can be restored.** Losing it means the off-site backups are worthless; check that it is still accessible as part of the quarterly test (12.7).

Ransomware safety. The script uses `rclone copy`, never `sync`, so a local deletion or wipe never propagates to the bucket. Old objects are expired by a bucket lifecycle rule, not by the script. A small budget alert is configured in the R2 dashboard.

Daily verification:

```
tail -20 /var/log/naro-offsite-backup.log      # last line: "OK off-site backup complete"
```

Healthchecks.io alerting (LIVE). Project "Naro Fashion" on the owner's Healthchecks.io account, email integration:

Check	Schedule (UTC)	Grace	Env var in `/etc/naro-backup.env`
`naro-pg-backup`	`15 3 * * *`	2 h	`HEALTHCHECK_PGBACKUP_URL`
`naro-offsite-backup`	`0 4 * * *`	2 h	`HEALTHCHECK_BACKUP_URL`

A missed run or a `/fail` ping emails the owner. Test a check with `sudo /usr/local/bin/naro-pg-backup.sh` — it should turn green within seconds.

Setting it up on a new server: follow `docs/OPS/BACKUPS.md` "Off-site sync — operator setup" (written for Vultr Object Storage; for R2 use `BACKUP_S3_PROVIDER=Cloudflare` and the `https://<account-id>.r2.cloudflarestorage.com` endpoint, an R2 API token scoped to *Object Read & Write* on this bucket only, and keep `NO_CHECK_BUCKET` behaviour — the script already sets it because object-scoped tokens cannot create buckets). Then create the cron file:

```
cat > /etc/cron.d/naro-offsite-backup <<'EOF'
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
MAILTO=root
# 04:00 UTC — 45 min after the 03:15 pg dump
0 4 * * * root APP_DIR=/var/www/naro-fashion /usr/local/bin/naro-offsite-backup.sh
EOF
chmod 0644 /etc/cron.d/naro-offsite-backup
/usr/local/bin/naro-offsite-backup.sh && tail -20 /var/log/naro-offsite-backup.log
```

12.5 Restoring the Database

Use a local dump (`/var/backups/naro/postgres/`) when the server is intact; otherwise download and decrypt an off-site copy first (12.6).

```
# 1. Stop writers (Naro processes only)
pm2 stop naro-api naro-admin naro-storefront

# 2. Optional but recommended: dump the current state first
BACKUP_TAG=before-restore bash /var/www/naro-fashion/scripts/ops/pg-backup.sh

# 3. Drop + recreate the database
sudo -u postgres psql -c "DROP DATABASE naro_fashion;"
sudo -u postgres psql -c "CREATE DATABASE naro_fashion OWNER naro_admin;"

# 4. Restore (substitute the dump file)
PGPASSFILE=/root/.pgpass pg_restore --clean --if-exists --no-owner --no-acl \
  -h localhost -U naro_admin -d naro_fashion \
  /var/backups/naro/postgres/naro_fashion-YYYY-MM-DD_HHMMSSZ.dump

# 5. Re-apply table ownership (Part 14.9) if anything was created as postgres

# 6. Make sure the code matches the schema of the dump. If the dump predates a
#    migration that the current code needs, roll the code back too
#    (bash scripts/ops/rollback.sh --yes, or check out + redeploy the matching commit).
cd /var/www/naro-fashion/packages/database && pnpm exec prisma migrate status

# 7. Bring the Naro apps back
pm2 restart naro-api naro-admin naro-storefront
```

Smoke test: `curl -s https://api.narofashion.co.tz/api/v1/health` must report `"db":"ok"`, then log in to the admin and check recent orders.

> *`deploy.sh` and `scripts/ops/deploy-lib.sh` print a ready-made version of these commands (with the exact pre-deploy dump path) whenever a deploy fails. Note that their printed text ends with `pm2 restart all` — on this shared VPS use `pm2 restart naro-api naro-admin naro-storefront` instead.*

12.6 Rebuilding the Server from the Off-site Backup

Use this if the VPS is lost. You need: the GitHub repo, the `age` private key (owner's password manager), and an R2 API token with **read** access to the bucket (create a temporary read-only token in the Cloudflare dashboard — the server's token is write-scoped).

1. Provision a new VPS and do Part 1 and Part 2.1–2.4, 2.7 (**not** 2.5/2.6 — the restored `.env` files carry the DB password and secrets). Verify and record the new host-key fingerprints (Part 1.3) and **update the `fingerprint:` in `deploy-prod.yml`** (Part 11.3).
2. Clone the `prod` branch to `/var/www/naro-fashion` (Part 3.1).
3. Fetch and decrypt the newest server-config bundle and DB dump (on the new VPS or your laptop):

```
apt install -y rclone age
# configure an rclone remote "r2" for the bucket with the temporary read-only token
# (rclone config, provider Cloudflare, endpoint https://<account-
id>.r2.cloudflarestorage.com)
rclone lsl r2:<bucket>/server-config | sort -k2,3 | tail -3
rclone copy r2:<bucket>/server-config/<newest>.tar.gz.age .
rclone copy r2:<bucket>/postgres/<newest>.dump.age .
```

```
age -d -i naro-backup-age.key -o server-config.tar.gz <newest>.tar.gz.age
age -d -i naro-backup-age.key -o restore.dump        <newest>.dump.age
tar -tzf server-config.tar.gz            # inspect first
sudo tar -xzf server-config.tar.gz -C /  # restores the .env files, /etc/nginx,
/etc/letsencrypt, backup cron files
```

Copy the age key onto the server only for the duration of the restore, then **delete it** (`shred -u naro-backup-age.key`) — it must not live on the VPS.

4. Create the DB user and database with the password from the restored `packages/database/.env` (Part 2.5), then restore `restore.dump` (12.5 steps 3–5).
5. Restore files: `rclone copy r2:<bucket>/uploads /var/www/naro-fashion/apps/api/uploads` , and decrypt + untar the newest `private-uploads/*.tar.gz.age` into `/var/www/naro-fashion/apps/api/` (it contains the `private-uploads/` directory).
6. `pnpm install --frozen-lockfile` , `prisma generate` , then Part 3.6 (backups — `setup-backups.sh` re-creates `.pgpass`), Part 3.7 (first build + PM2 start), `nginx -t && systemctl reload nginx` (restored config + certificates), point DNS at the new IP (Part 7), and run `bash deploy.sh` once (Part 15.2).
7. `TWO_FA_ENCRYPTION_KEY` **must be restored exactly** (it is in the restored `apps/api/.env`). A new value makes every enrolled admin's 2FA unusable.
8. Revoke the temporary read-only R2 token; reinstate off-site backups (12.4) and Healthchecks.

12.7 Quarterly Restore Test (MANUAL — and record it)

A backup that has never been restored is a hope, not a backup. Once a quarter, on a laptop or throwaway machine (never the production DB):

```
rclone copy r2:<bucket>/postgres/<newest>.dump.age .
age -d -i naro-backup-age.key -o restore.dump <newest>.dump.age
createdb naro_restore_test
pg_restore --no-owner --no-acl -d naro_restore_test restore.dump
psql -d naro_restore_test -c 'SELECT count(*) FROM "Product";' -c 'SELECT count(*) FROM
"Order";' -c 'SELECT max("createdAt") FROM "Order";'
# Also: decrypt and list one private-uploads archive and one server-config bundle
age -d -i naro-backup-age.key private-uploads-<ts>.tar.gz.age | tar -tz | head
dropdb naro_restore_test && rm -f restore.dump
```

Record each test (date, dump used, row counts, who) in the table in `docs/OPS/BACKUPS.md` "Restore test". Last recorded: **2026-10-18** — encrypted off-site dump decrypted with the owner's key and fully restored to SQL.

12.8 PM2 Log Rotation

```
pm2 install pm2-logrotate

# Configure: max 10MB per log file, keep 7 rotated files
pm2 set pm2-logrotate:max_size 10M
pm2 set pm2-logrotate:retain 7
pm2 set pm2-logrotate:compress true
```

12.9 Security Features Already Configured

The application has these security measures built in (details in `docs/SECURITY.md` and the project `CLAUDE.md`):

Feature	Details
Helmet + CSP	HTTP security headers on the API; both Next apps send a per-request **nonce CSP** from `middleware.ts`
Rate limiting	Global throttler: 100 req/min default; auth routes 3–5/min; `trust proxy` = 1 (behind Nginx)
Account lockout	5 failed logins → 30-minute lock, for tenant admins and platform admins
TOTP 2FA	For AdminUser and PlatformAdmin; needs `TWO_FA_ENCRYPTION_KEY`
JWT hygiene	Separate access/refresh secrets (enforced at boot), token `typ` + `tokenVersion` — logout/password change kills older tokens
CORS	Only `STOREFRONT_URL` (comma-separated list) and `ADMIN_URL` origins
Tenant isolation	`tenantId` on every query; global `TenantGuard` (403 for suspended tenants); runtime tenant-scope guard (`TENANT_SCOPE_ENFORCEMENT`)
Payment webhooks	Selcom fails closed in production when unconfigured; HMAC verified over the raw body
Private ID documents	Stored in `PRIVATE_UPLOAD_DIR` (default `apps/api/private-uploads/`), never served statically, backed up encrypted
Swagger	Off in production unless `ENABLE_SWAGGER=true`

12.10 Monitoring and Remaining Improvements

- **Live:** Healthchecks.io dead-man's switches for both backup jobs (12.4); deploy health checks + red CI on failure (Part 11).
- **Recommended, not yet confirmed live:** UptimeRobot (free) monitors on the storefront, admin and `https://api.narofashion.co.tz/api/v1/health` with SSL-expiry alerts; Sentry error tracking. Setup steps: `docs/OPS/MONITORING.md`.
- **Hardening backlog:** `PasswordAuthentication no` (12.2), fail2ban for SSH, `TENANT_SCOPE_ENFORCEMENT=strict` once `pm2 logs naro-api | grep TENANT_SCOPE` is clean.

Part 13: Post-Deployment Verification

Run through this after every deployment. **Verify live behaviour, not just the commit.** Matching `git rev-parse HEAD` on the VPS only proves the working tree changed — not that PM2 is serving the new build. The strongest evidence is hitting something the release actually changed.

13.1 Service Health Checks

```
# 1. Which commit is the API running? (GIT_SHA set by deploy.sh)
curl -s https://api.narofashion.co.tz/api/v1/health
# Expected: {"status":"ok","db":"ok","commit":"<sha of the prod tip>", ...}
git log origin/prod --oneline -1          # on your machine — compare the SHA

# 2. Storefront and admin answer
curl -s -o /dev/null -w "%{http_code}\n" https://narofashion.co.tz          # 200
```

```
curl -s -o /dev/null -w "%{http_code}\n" https://admin.narofashion.co.tz    # 200 or 3xx to
/login

# 3. A real static asset is served (an HTML 200 is NOT enough — Part 4.13)
asset=$(curl -s https://narofashion.co.tz | grep -oE '/_next/static/[^"\\ ]+\.css' | head
-1)
curl -s -o /dev/null -w "%{http_code}\n" "https://narofashion.co.tz${asset}"   # 200

# 4. On the server: processes and deploy state
pm2 list                                  # naro-* online, low restart counts
cat /var/www/naro-fashion/.deploy/current_sha /var/www/naro-fashion/.deploy/previous_sha

# 5. TLS
curl -vI https://narofashion.co.tz 2>&1 | grep -E "expire date|subject:"
```

Then **exercise the change**: the endpoint the commit added, an admin route that should now reject a customer token, a page whose layout changed, etc.

13.2 Full Verification Checklist (first install / after a rebuild)

#	Task	How to Verify	Expected Result
1	Storefront loads	Visit `https://narofashion.co.tz`	Homepage renders **styled**, with products
2	Admin loads	Visit `https://admin.narofashion.co.tz`	Login page renders
3	API healthy	`https://api.narofashion.co.tz/api/v1/health`	`status: ok`, `db: ok`, correct `commit`
4	SSL working	Padlock in browser	Valid certificate on all domains
5	Admin login	Tenant admin account (Part 3.5)	Dashboard loads
6	Platform login	`https://admin.narofashion.co.tz/platform-login`	Platform dashboard loads
7	**CHANGE PASSWORDS + enable 2FA**	Both seeded admin accounts	New password works; 2FA prompt on next login
8	Products + images	Browse the shop	Images render (no broken images)
9	Email	"Forgot password" on the storefront	Email arrives (Brevo)
10	Mobile works	Phone or responsive mode	Mobile layout correct
11	Backups	`ls /etc/cron.d/naro-*`, `tail /var/log/naro-pg-backup.log /var/log/naro-offsite-backup.log`	Both cron files present, last runs OK, Healthchecks green
12	Swap enabled	`free -h`	Swap ~2.0G

13	PM2 auto-start	Reboot in a quiet window, wait ~60 s	`naro-*` (and the other apps) back online
14	Firewall active	`ufw status`	SSH + Nginx Full only
15	Rollback ready	`ls apps/*/.next-prev apps/api/dist-prev; cat .deploy/previous_sha`	Present after the second successful deploy

13.3 Test Authentication

```
# Tenant admin login via API (use the real password; it is not printed here)
curl -s -X POST https://api.narofashion.co.tz/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@narofashion.co.tz","password":"<admin_password>"}' | head -c 300
# Expected: JSON with tokens — or { "requires2FA": true, "challengeToken": ... } when 2FA is
on

# An unauthenticated admin endpoint must be rejected
curl -s -o /dev/null -w "%{http_code}\n" https://api.narofashion.co.tz/api/v1/admin-users
# 401
```

Avoid putting real passwords on the command line on shared machines (shell history) — prefer testing logins in the browser.

Part 14: Known Issues and Solutions

These are real issues encountered during deployment and ongoing operation, with proven solutions.

14.1 TenantContext for @Public() Endpoints

Problem: Endpoints decorated with `@Public()` skip `JwtAuthGuard`, so `req.user` is never populated. But these endpoints still need the `tenantId` for data scoping (e.g., `GET /products`, `GET /cms/pages`).

Solution (already implemented): Both `TenantContext` and `ModuleGuard` have a JWT fallback. When `req.user` is null, they read the JWT from the `Authorization` header directly. `TenantContext` **verifies** the token signature (with `JWT_SECRET`) and rejects with 403 when an `X-Tenant-Id` header disagrees with the token's tenant; `ModuleGuard` does an unverified payload decode by design (advisory — the `TenantModule` table lookup is the real gate). If CMS pages or product listings start failing for logged-in admins, check this fallback — and check that `JWT_SECRET` is consistent across the three `.env` files.

14.2 ModuleGuard DI Issues

Problem: `ModuleGuard` originally tried to inject `JwtService` for token decoding. This caused dependency injection failures in non-auth modules because `JwtService` is only provided by `AuthModule`.

Solution (already implemented): `ModuleGuard` uses manual Base64 decoding of the JWT payload (`Buffer.from(payload, 'base64url').toString()`) instead of `JwtService`. No external dependency needed.

14.3 API Crash-Looping (Restart Count > 1000)

Symptom: `pm2 list` shows `naro-api` with a restart count climbing into hundreds or thousands.

Diagnosis:

```
# Check the error
pm2 logs naro-api --lines 50

# Or run the API directly to see the full error:
cd /var/www/naro-fashion/apps/api
node dist/main.js
```

Common causes:

Error	Fix
`Cannot find module '…'`	Incomplete install — redeploy (`pnpm install --frozen-lockfile`); never `pnpm add` on the VPS (Part 4.4)
`Cannot find module '@prisma/client'` / `.prisma/client`	`cd packages/database && pnpm exec prisma generate`
`ECONNREFUSED 127.0.0.1:5432`	PostgreSQL not running: `systemctl start postgresql`
`Refusing to start: JWT_SECRET …` / `JWT_SECRET and JWT_REFRESH_SECRET must be different`	Env problem — Part 14.12
Prisma `column … does not exist`	Code is ahead of the schema — a migration was not applied. `pnpm exec prisma migrate status` (Part 15.4)
DI injection error (NestJS)	Missing provider in a module — check the error message for the class name

If the crash started with a deploy, the automatic rollback normally restores the previous build; otherwise `bash scripts/ops/rollback.sh --yes` (Part 15.5). After fixing, restart:

```
pm2 restart naro-api
pm2 reset naro-api   # Resets the restart counter to 0
```

14.4 502 Bad Gateway After Restart

Symptom: After a deploy or a restart of the Naro apps, visiting any URL shows "502 Bad Gateway" from Nginx.

Cause: Applications take 10-15 seconds to start up (one instance each, so there is no overlap — Part 5.1). The API is the slowest because it initializes Prisma connections, seeds roles/permissions, and starts cron jobs.

Solution: Wait 15 seconds and try again. If it persists after 30 seconds, check `pm2 list` — one or more apps may have crashed (status = `errored` or high restart count).

14.5 Storefront Shows "Store not found"

Symptom: Storefront loads but shows a "Store not found" or blank error page.

Causes:

1. **Tenant domain not set** — The `Tenant` table has no `domain` value, or it does not match the URL you are visiting. Fix with Part 10.
2. **DNS not pointing to server** — The domain resolves to a different IP. Check with `dig yourdomain.com +short`.
3. **API not running** — The storefront middleware calls the API to resolve the tenant. If the API is down, tenant resolution fails. Check `pm2 list`.
4. **Host not allowed** — in production, unknown hosts get a 404. Check the Host is the tenant's `domain` (or `www.` of it) or listed in `STOREFRONT_DEFAULT_HOSTS` (Part 3.2).
5. **Tenant suspended** — `SUSPENDED` / `DEACTIVATED` tenants get 403 `TENANT_SUSPENDED` from the API (subscription lapsed). Check `status` in the query below.

Fix:

```
# Check the tenant domain in the database
sudo -u postgres psql -d naro_fashion -c 'SELECT id, name, slug, domain, status FROM
"Tenant";'

# Set/fix the domain
sudo -u postgres psql -d naro_fashion -c "UPDATE \"Tenant\" SET \"domain\" =
'yourdomain.com' WHERE \"slug\" = 'naro-fashion';"

# Restart storefront
pm2 restart naro-storefront
```

14.6 "Tenant context is required but not available"

Symptom: API returns **400** with a message asking to "Provide X-Tenant-Id header or a valid Authorization token" (older builds returned 500 "Tenant context is required but not available").

Cause: An admin user is calling an endpoint that requires `tenantId`, but the request does not have the `X-Tenant-Id` header and the JWT does not contain a `tenantId`. This can happen with:

- Public endpoints called without authentication
- Platform admin calling tenant-scoped endpoints

Solution: This is already handled by the JWT decode fallback in `TenantContext`. If it still occurs, ensure the admin user has a `tenantId` in their JWT (check by decoding the token at https://jwt.io).

14.7 Static Assets Not Loading (Unstyled / Blank Page)

Symptom: The page loads as unstyled HTML or blank. Browser console shows 404 errors for `/_next/static/...` `.js` and `.css` files.

Causes, most likely first:

1. Static files were not copied into the standalone folder after a manual build → copy them (Part 4.7) and `pm2 restart naro-storefront naro-admin`.
2. A Next build was produced in a different `distDir` and renamed (Part 4.13) → rebuild in place: `pnpm --filter <app> build`, copy static, restart.
3. A worker kept serving from a moved directory after a graceful reload (Part 4.14) → `pm2 restart naro-storefront naro-admin`.
4. Fastest safe recovery when unsure: `bash scripts/ops/rollback.sh --yes` (Part 15.5).

14.8 Deploy Fails at the git Step (untracked or drifted files)

Symptom: the deploy log shows `error: The following untracked working tree files would be overwritten by merge` or `error: Your local changes to the following files would be overwritten by merge: pnpm-lock.yaml`. `gh run view <id> --log-failed` surfaces it immediately.

Causes:

- **Untracked files** were `scp`'d into `/var/www/naro-fashion/` at paths a later commit now tracks.
- **Tracked-file drift**: someone ran `pnpm install` / `pnpm add` manually on the VPS (rewrites `pnpm-lock.yaml`), or edited `ecosystem.config.js` /a script in place.

Fix:

```
cd /var/www/naro-fashion
git status --short                       # see what is in the way
rm -f <untracked paths that the commit now tracks>
git checkout -- pnpm-lock.yaml           # the committed lockfile is authoritative
EXPECTED_SHA=$(git rev-parse origin/prod) bash deploy.sh    # or re-run the CI job
```

Prevention: put one-off files in `/tmp/`; never run `pnpm install` on the VPS (only `deploy.sh`, which uses `--frozen-lockfile`); change tracked files only through git. (`.env` files are git-ignored and are never affected.)

14.9 PostgreSQL Ownership (`must be owner of table`)

Run once per database — on a new DB whose tables were created by `postgres`, or after a restore run as a different user:

```
sudo -u postgres psql -d naro_fashion <<'SQL'
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO naro_admin;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO naro_admin;
DO $$
DECLARE r RECORD;
BEGIN
  FOR r IN SELECT tablename FROM pg_tables WHERE schemaname = 'public' LOOP
    EXECUTE format('ALTER TABLE public.%I OWNER TO naro_admin', r.tablename);
  END LOOP;
  FOR r IN SELECT sequence_name FROM information_schema.sequences WHERE sequence_schema =
'public' LOOP
    EXECUTE format('ALTER SEQUENCE public.%I OWNER TO naro_admin', r.sequence_name);
  END LOOP;
END $$;
SQL
```

Objects created afterwards by migrations (which connect as `naro_admin`) are owned correctly.

14.10 Admin Lost Their 2FA Device

If an admin has neither their authenticator nor a recovery code, reset 2FA in the database (also invalidates all their tokens):

```
sudo -u postgres psql -d naro_fashion -c "UPDATE \"AdminUser\" SET \"is2FAEnabled\"=false,
\"twoFASecret\"=NULL, \"twoFARecoveryCodes\"='{}', \"tokenVersion\"=\"tokenVersion\"+1 WHERE
email='<admin_email>';"
```

Ask them to re-enrol immediately. If **every** admin's 2FA broke at once, suspect a changed `TWO_FA_ENCRYPTION_KEY` (or a mismatch between the three `.env` files) — restore the original value rather than resetting everyone.

14.11 Env Change "Doesn't Take Effect" (CORS, secrets, URLs)

Symptom: you edited an env var and restarted, but the API still behaves as before (e.g. `www.` gets no CORS header).

Cause: the dotenv precedence trap (Part 3.2) — `packages/database/.env` still has the old value and wins.

Fix: make the key identical in all three backend `.env` files, then `pm2 restart naro-api --update-env`. To see the real runtime value without guessing, check what the process was started with (`pm2 env <id>` for PM2-provided vars) or temporarily log it from the app. `NEXT_PUBLIC_*` values need a rebuild (a deploy), not a restart.

14.12 API Refuses to Boot After an Env Change

Log message	Cause / fix
`Refusing to start: JWT_SECRET env var must be set (>=32 chars) in production` (or `JWT_REFRESH_SECRET`)	Missing/short secret — often an empty value in `packages/database/.env` winning. Fix all three files
`Refusing to start: JWT_SECRET and JWT_REFRESH_SECRET must be different values in production`	Generate a new `JWT_REFRESH_SECRET` with `openssl rand -hex 48` (logs everyone out once)

Pre-flight before any auth-touching deploy: `grep -c '^JWT_SECRET=' /var/www/naro-fashion/apps/api/.env` must return ≥ 1 and the value must be ≥ 32 chars. A boot failure after the swap triggers the automatic rollback — fix the env, then redeploy.

14.13 Deploy Aborted on a Migration Problem

Log message	Meaning / fix
`DRIFT: the live database differs from the 0_init baseline`	One-time baseline found differences (5a). Nothing was changed. Reconcile by hand — Part 15.4
`schema.prisma has changes with NO migration`	Someone edited `schema.prisma` without `prisma migrate dev`. Old build still live. Create the migration locally, commit, push
`prisma migrate deploy` error, migration marked **failed**	Blocks every later deploy until resolved — Part 15.4 "A migration failed"
`pre-deploy pg_dump failed — refusing to touch the schema`	`/root/.pgpass` missing/wrong, or disk full (`df -h /`). Check `/var/log/naro-pg-backup.log`; re-run `setup-backups.sh`

Part 15: Updating the Application

Golden rules for the shared 2GB production box:

- Deploy by pushing to `prod`. Manual runs use `deploy.sh`, never hand-rolled build/restart commands.

- **Never** root `pnpm build` (parallel → OOM), **never** `pm2 restart all` (restarts other apps), **never** `pnpm install` by hand on the VPS (lockfile drift), **never** `prisma db push` against production.
- Never deploy while another application on the VPS is building.

15.1 Via CI/CD (Recommended)

```
git checkout master
# ... make changes, commit ...
git push origin master

git checkout prod
git merge master --no-edit
git push origin prod    # verify → deploy (Part 11)

gh run list --workflow "Verify & Deploy to Production" --limit 1
git checkout master
```

Then verify (Part 13.1).

15.2 Manual Deploy on the VPS

When CI is unavailable, or to re-run a deploy after fixing a server-side problem (e.g. Part 14.8):

```
ssh -o StrictHostKeyChecking=yes root@80.240.30.107
cd /var/www/naro-fashion
git fetch origin prod
EXPECTED_SHA=$(git rev-parse origin/prod) bash deploy.sh
```

Passing `EXPECTED_SHA` keeps the commit assertion on. The run takes ~10–20 minutes (mostly the three builds); watch for the final ✅ `Deployment complete` line. The same guards apply: pre-deploy dump, migrations, health checks, automatic rollback.

> *The old "manual update" procedure (`git pull origin master` , `pnpm install` , `prisma db push --accept-data-loss` , `pnpm build` , `pm2 restart all`) is* ***obsolete and dangerous*** *— it pulls the wrong branch, can drop data, OOMs the box and restarts other tenants' apps. Do not use it.*

15.3 Schema Changes (migrations, expand/contract)

The schema only changes through committed migrations. Workflow (on your development machine, from `packages/database/` , using the local Prisma CLI):

1. Edit `packages/database/prisma/schema.prisma` .
2. Generate the migration:

   ```
   cd packages/database
   pnpm prisma migrate dev --name add_order_gift_note
   ```

   This writes `prisma/migrations/<timestamp>_add_order_gift_note/migration.sql` , applies it locally and regenerates the client.
3. **Read the generated SQL** and fix it by hand if needed (e.g. add a backfill `UPDATE` before a `SET NOT NULL`).

4. Commit `schema.prisma` **and** the migration folder in the same commit. CI's migrations-vs-schema check fails if you forget the migration.
5. Push to `prod` . `deploy.sh` takes a pre-deploy dump, runs `prisma migrate deploy` , then the drift guard.

Expand / contract — mandatory for destructive changes. Migrations are forward-only and rollbacks (automatic or manual) do **not** roll the database back, so the *previous* release must keep working against the *new* schema:

- **Additive changes ship in one release:** new table, new nullable column, new column with a default, new index.
- **Destructive or narrowing changes take two releases:**
  1. *Expand* — add the new column/table, write to both, backfill; code stops reading the old column. Deploy, let it settle.
  2. *Contract* — a later release drops/renames the old column, adds `NOT NULL` , tightens a unique constraint.
- Renames are add-new + copy + drop-old across two releases, never `RENAME` in one.
- New `@@unique` constraints: first remove duplicates (script or migration), then add the constraint.
- Anything Prisma flags as data loss in `migrate dev` must be reviewed and split before it is committed.
- Never edit a migration that has already been applied in production — add a new one.

Local dev databases created with `db push` must be baselined once (`pnpm prisma migrate resolve --applied 0_init` after confirming no diff) — see `docs/OPS/MIGRATIONS.md` .

15.4 When a Migration Step Fails in Production

Baseline drift (first deploy on a legacy DB only). The log prints the SQL that would make the live DB match `0_init` . Nothing was pushed. Read it; if prod is simply behind, apply that SQL deliberately after review (or restore a known-good dump); if prod has extra hand-made objects, decide whether they belong in `schema.prisma` . Re-run `deploy.sh` once `migrate diff` reports no difference.

A migration failed. `migrate deploy` aborts before building (old build stays live) and records the migration as **failed**, which blocks every later deploy:

```
cd /var/www/naro-fashion/packages/database
pnpm exec prisma migrate status
# If the failed migration left nothing behind (rolled back by its transaction):
pnpm exec prisma migrate resolve --rolled-back <migration_folder_name>
# If it partly applied: finish or undo the remaining statements by hand, then
#   pnpm exec prisma migrate resolve --applied <name>     (finished)
#   pnpm exec prisma migrate resolve --rolled-back <name> (undone)
```

Then fix the migration in git (or add a corrective one) and redeploy. Last resort: restore the pre-deploy dump printed in the deploy log (Part 12.5).

15.5 Rolling Back a Bad Release

If a deploy went live but turns out to be broken (the automatic rollback only covers failed health checks):

```
ssh -o StrictHostKeyChecking=yes root@80.240.30.107 "cd /var/www/naro-fashion && bash
scripts/ops/rollback.sh --yes"
```

What it does (same helpers as the automatic rollback):

1. refuses unless `.deploy/previous_sha` is set and `apps/storefront/.next-prev`, `apps/admin/.next-prev` and `apps/api/dist-prev` all exist — it never mixes builds from different commits;
2. swaps `*-prev` back into place (bad builds kept as `*-failed`);
3. `git reset --hard <previous_sha>` (+ `pnpm install --frozen-lockfile` if the lockfile differs, + `prisma generate`);
4. reloads the API / restarts the Next apps, runs the health checks.

It is **one-shot**: the `*-prev` dirs are consumed, so a second run refuses until the next successful deploy. The **database is not touched**.

Afterwards, revert the bad commit on `prod` (`git revert <sha>` on `prod`, push) — otherwise the next push to `prod` redeploys it. Without `--yes` the script asks for confirmation (interactive terminals only).

15.6 Data-Change Scripts (backfills, one-off repairs)

`deploy.sh` deploys **code and schema only**. It never runs seeds, imports or backfills. Data scripts are committed under `scripts/` and run by hand, once, on the VPS:

```
cd /var/www/naro-fashion
# 1. Ad-hoc dump first
BACKUP_TAG=pre-$(date -u +%H%M) bash scripts/ops/pg-backup.sh
# 2. Dry run when the script supports it, read the output
node scripts/restore-size-guide-tables.js naro-fashion --dry-run
# 3. Real run
node scripts/restore-size-guide-tables.js naro-fashion
```

Examples in `scripts/`: `restore-size-guide-tables.js <tenantSlug> [--dry-run]`, `backfill-variant-tenant.js`, `backfill-product-images.js`, `backfill-instagram-images.js`, `instagram-connect.js <tenantSlug>`. Read each script's header for its arguments and whether it supports `--dry-run` before running it.

Gotchas:

- Run scripts **from the repo** (they resolve Prisma via `packages/database/node_modules/@prisma/client`). Do not run Prisma scripts from `/tmp` — pnpm's directory-dependent resolution will not find the client.
- `scripts/bulk-import-products.js` reads photos from git-ignored `docs/photo-staging/` and fails on the VPS. Commit the destination images under `apps/api/uploads/products/` instead, then run `scripts/backfill-product-images.js` on the VPS (idempotent).
- Do not `scp` helper scripts into the repo directory (Part 14.8) — use `/tmp/` for anything uncommitted that does not need Prisma.

15.7 Environment-Only Changes

If you only changed environment variables (no code):

```
cd /var/www/naro-fashion
# Edit the SAME key in all three backend files (don't blindly cp one over the
# others unless you have confirmed they are identical — `diff .env apps/api/.env`)
nano .env; nano apps/api/.env; nano packages/database/.env
grep -H '^STOREFRONT_URL=' .env apps/api/.env packages/database/.env   # example consistency
check
```

```
pm2 restart naro-api --update-env
curl -s http://127.0.0.1:4000/api/v1/health
```

`NEXT_PUBLIC_*` and other frontend build-time values require a deploy (rebuild), not a restart. The nightly server-config backup picks up the new `.env` automatically.

Part 16: Cost Summary

16.1 Monthly Costs

Service	Provider	Monthly Cost	Notes
VPS	Vultr (vhf-1c-2gb)	$12.00	1 vCPU, 2GB RAM, 64GB NVMe, Frankfurt — shared with other applications
Domain	Habari Node (.co.tz)	~$1.76	TZS 21,186/year (~$8.25/year)
SSL	Let's Encrypt	**Free**	Auto-renews every 90 days
Email	Brevo (free tier)	**Free**	300 emails/day
DNS	Vultr DNS	**Free**	Included with VPS
CI/CD	GitHub Actions	**Free**	Free-plan minutes; verify + deploy ≈ 15–25 min per push to `prod`
Off-site backups	Cloudflare R2	**Free**	10 GB free tier (~65 MB used at go-live); $1 budget alert set
Backup alerting	Healthchecks.io	**Free**	Hobbyist plan (20 checks)
TOTAL		**~$13.76/mo**	**~$152.25/year**

16.2 Scaling Costs

If you outgrow the current server:

Plan	Specs	Cost	When to upgrade
Current (vhf-1c-2gb)	1 vCPU, 2GB RAM	$12/mo	Up to ~1,000 concurrent users
vhf-2c-4gb	2 vCPU, 4GB RAM	$24/mo	Heavy traffic, many tenants
vhf-4c-8gb	4 vCPU, 8GB RAM	$48/mo	10+ tenants, analytics workloads

16.3 Adding a New Tenant Client

When selling the platform to a new client:

1. **Login** as platform admin at `https://admin.yourdomain.com/platform-login`
2. Go to **Tenants** > **New Tenant**

3. Fill in: company name, slug, admin email/password, subscription plan
4. The new tenant gets their own isolated storefront, admin, and data

To connect a **custom domain** for the new tenant:

```
# 1. Add their domain as an A record in DNS pointing to your VPS IP
# 2. Get SSL for the new domain:
certbot --nginx -d newclient.co.tz

# 3. Set the domain in the database:
sudo -u postgres psql -d naro_fashion \
  -c "UPDATE \"Tenant\" SET \"domain\" = 'newclient.co.tz' WHERE \"slug\" = 'new-client-slug';"

# 4. Restart storefront:
pm2 restart naro-storefront
```

Two extra steps that are easy to forget:

- **Nginx must know the domain before certbot runs.** Add the new domain (and its `www.`) to the storefront `server_name` in `/etc/nginx/sites-available/narofashion` (or a new server block proxying to `127.0.0.1:3000`), then `nginx -t && systemctl reload nginx`, then run certbot.
- **CORS.** The API only allows origins listed in `STOREFRONT_URL` / `ADMIN_URL`. Append `https://newclient.co.tz,https://www.newclient.co.tz` to `STOREFRONT_URL` in **all three** backend `.env` files (Part 3.2) and `pm2 restart naro-api --update-env` — otherwise the new storefront's browser requests fail with CORS errors.

Appendix A: All Environment Variables Reference

Values shown are examples or defaults — **never** real secrets. Real values live only in the server's `.env` files (backed up encrypted, Part 12.4) and the owner's password manager.

Backend `.env` — root, `apps/api/.env` and `packages/database/.env` (keep all three identical)

Core (required in production):

Variable	Required	Example / default	Description
`NODE_ENV`	Yes	`production`	Enables production-only guards (JWT secret checks, Selcom fail-closed, Swagger off, unknown-host 404)

`DATABASE_URL`	Yes	`postgresql://naro_admin:<db_password>@localhost:5432/naro_fashion?schema=public`	PostgreSQL connection string (also used by Prisma CLI / migrations)
`JWT_SECRET`	Yes	`openssl rand -hex 48`	Access-token signing key. **≥ 32 chars or the API refuses to boot**
`JWT_REFRESH_SECRET`	Yes	`openssl rand -hex 48`	Refresh-token signing key. **≥ 32 chars and must differ from `JWT_SECRET`** or the API refuses to boot
`STOREFRONT_URL`	Yes	`https://narofashion.co.tz,https://www.narofashion.co.tz`	CORS origins (comma-separated) + links in emails
`ADMIN_URL`	Yes	`https://admin.narofashion.co.tz`	CORS origin for the admin app
`TWO_FA_ENCRYPTION_KEY`	Yes, to enable 2FA	`openssl rand -hex 32`	Encrypts admin TOTP secrets (≥ 32 chars). **Never rotate after enrolment.** Without it the API boots but 2FA cannot be enabled

Auth / session:

Variable	Default	Description
JWT_ACCESS_EXPIRES	15m	Default access-token lifetime (per-tenant override: SiteSetting auth_access_token_expires)
JWT_REFRESH_EXPIRES	7d	Default refresh-token lifetime (per-tenant override: auth_refresh_token_expires)
TWO_FA_ISSUER	Naro Fashion	Issuer label shown in authenticator apps

> *The April 2026 guide listed `JWT_EXPIRATION` / `JWT_REFRESH_EXPIRATION` — those names are not read by the API. Use `JWT_ACCESS_EXPIRES` / `JWT_REFRESH_EXPIRES`.*

Tenancy, security and platform behaviour:

Variable	Default	Description
DEFAULT_TENANT_SLUG	falls back to NEXT_PUBLIC_TENANT_SLUG, then the sole tenant	Default tenant for env-based Instagram fallback and email branding
TENANT_SCOPE_ENFORCEMENT	warn	Runtime tenant-scope guard: warn (log once per call site) / strict (throw) / off (break-glass). Review pm2 logs naro-api | grep TENANT_SCOPE before switching to strict
TENANT_SCOPE_STRICT_BY_ID	unset	true makes strict mode also throw for id-only lookups
ENABLE_SWAGGER	unset (off in production)	true exposes Swagger docs in production — leave off
GIT_SHA	set by deploy.sh via ecosystem.config.js	Reported as commit by GET /api/v1/health. Do not set by hand
PRIVATE_UPLOAD_DIR	apps/api/private-uploads/	Private storage for national-ID documents (not served statically; backed up encrypted)
API_PUBLIC_URL	unset	Public API base (e.g. https://api.narofashion.co.tz) used to build absolute image URLs in emails
PORT	4000 (set by PM2)	API port

Email and SMS:

Variable	Example / default	Description
SMTP_HOST	smtp-relay.brevo.com	SMTP server (Part 9)
SMTP_PORT	587	SMTP port (STARTTLS)
SMTP_USER	<account-id>@smtp-brevo.com	SMTP login

`SMTP_PASS`	`<smtp_key>`	Brevo SMTP key
`SMTP_FROM`	`noreply@narofashion.co.tz`	Sender address
`ADMIN_NOTIFICATION_EMAIL` / `ADMIN_NOTIFICATION_PHONE`	unset	Global override for admin alert recipients (scheduler)
`AT_USERNAME` / `AT_API_KEY` / `AT_SENDER_ID`	`sandbox` / unset	Africa's Talking SMS. Without them SMS is logged only

Payments:

Variable	Description
`SELCOM_API_KEY`, `SELCOM_API_SECRET`, `SELCOM_VENDOR`, `SELCOM_BASE_URL`	Selcom gateway. **In production Selcom fails closed when these are unset** (no simulated success, webhooks rejected)
`CLICKPESA_TOKEN_TTL_MINUTES`	ClickPesa auth-token cache TTL (default 55). ClickPesa credentials themselves are per tenant in `PaymentMethod.integrationParams`, not env

Business rules, crons and retention (all optional):

Variable	Default	Description
`ORDER_PAYMENT_TTL_HOURS`	`24`	Unpaid non-COD online orders auto-cancel after this
`MAX_OPEN_COD_ORDERS`	`3`	Open cash-on-delivery orders allowed per customer
`RENTAL_HOLD_TTL_HOURS`	`48`	Unpaid rental holds auto-cancel after this
`ABANDONED_CART_EMAIL_ENABLED`	`false`	Abandoned-cart reminder emails
`PAGEVIEW_RETENTION_DAYS`	`400` (minimum 30)	Visitor-analytics retention
`IDEMPOTENCY_RETENTION_DAYS`	`30`	Idempotency-key pruning

Integrations (optional):

Variable	Description
`INSTAGRAM_BUSINESS_ACCOUNT_ID`, `INSTAGRAM_ACCESS_TOKEN`	Legacy env fallback for the **default tenant only**. The live token is a per-tenant Page token stored in SiteSettings via Admin → CMS → Instagram Posts → Connect
`FACEBOOK_APP_ID`, `FACEBOOK_APP_SECRET`	Facebook app used for the Instagram connect flow / token checks
`ANTHROPIC_API_KEY`	Enables the admin AI assistant
`AI_ASSISTANT_MODEL`	Override the assistant model id
`AI_ASSISTANT_DAILY_REQUEST_LIMIT`	Daily request cap (default 200)
`AI_ASSISTANT_API_BASE`	API base the assistant calls (default `http://localhost:4000/api/v1`)

`AI_PHASE_3_2_DEMOTION_ENABLED`	Break-glass: `false` skips the SUPER_ADMIN `ai-agent:approve` demotion at boot

Storefront `.env.local` (`apps/storefront/.env.local`)

Variable	Required	Example	Description
`NEXT_PUBLIC_API_URL`	Yes	`https://api.narofashion.co.tz/api/v1`	API URL (build-time; also feeds the CSP `connect-src`)
`NEXT_PUBLIC_TENANT_SLUG`	Yes	`naro-fashion`	Fallback tenant slug (local dev / when domain resolution fails)
`STOREFRONT_DEFAULT_HOSTS`	Yes in production	`narofashion.co.tz,www.narofashion.co.tz`	Host allowlist (unknown hosts 404 in production). **First entry = Host header used by `deploy.sh` health checks**

Admin `.env.local` (`apps/admin/.env.local`)

Variable	Required	Example	Description
`NEXT_PUBLIC_API_URL`	Yes	`https://api.narofashion.co.tz/api/v1`	API URL (build-time)

Backup configuration — `/etc/naro-backup.env` (root, mode 600, NOT in the repo)

Read by `pg-backup.sh` and `offsite-backup.sh`. Values are in the file on the server and the owner's password manager only.

Variable	Description
`BACKUP_S3_PROVIDER`	rclone S3 provider — `Cloudflare` for R2
`BACKUP_S3_ENDPOINT`	R2 account endpoint
`BACKUP_S3_BUCKET`	Bucket name
`BACKUP_S3_ACCESS_KEY_ID` / `BACKUP_S3_SECRET_ACCESS_KEY`	Bucket-scoped R2 API token (Object Read & Write)
`BACKUP_AGE_RECIPIENT`	`age` **public** key. The private key never goes on the VPS
`HEALTHCHECK_PGBACKUP_URL`	Healthchecks ping URL for the nightly dump
`HEALTHCHECK_BACKUP_URL`	Healthchecks ping URL for the off-site sync

Appendix B: Quick Reference Commands

Deploy, Rollback and Health

```
ssh -o StrictHostKeyChecking=yes root@80.240.30.107        # trusted network only
cd /var/www/naro-fashion
EXPECTED_SHA=$(git rev-parse origin/prod) bash deploy.sh    # manual deploy (after git fetch
origin prod)
bash scripts/ops/rollback.sh --yes                         # one-shot rollback to the
previous build
curl -s http://127.0.0.1:4000/api/v1/health                # status, db, commit
cat .deploy/current_sha .deploy/previous_sha               # live / rollback-target commits
gh run list --workflow "Verify & Deploy to Production" --limit 3   # (local machine)
gh run view <run-id> --log-failed                                  # (local machine)
```

PM2 (Process Manager) — Naro processes only

```
pm2 list                                  # All processes (Naro + other apps on this VPS)
pm2 jlist                                 # Source of truth: script, cwd, exec mode, env
pm2 logs naro-api --lines 50 --nostream   # Last 50 lines of API logs
pm2 restart naro-api                      # Restart just the API
pm2 restart naro-storefront naro-admin    # Restart the Next apps
pm2 restart naro-api --update-env         # Restart and re-read PM2-provided env
pm2 info naro-api                         # Detailed process info
pm2 reset naro-api                        # Reset restart counter
pm2 monit                                 # Live CPU/memory monitor
pm2 save                                  # Save current process list
# NEVER on this server: pm2 restart all | pm2 stop all | pm2 delete all | pm2 flush (no
name)
```

Backups

```
ls -lh /var/backups/naro/postgres/ | tail -5                  # recent dumps
tail -20 /var/log/naro-pg-backup.log                          # nightly dump log
tail -20 /var/log/naro-offsite-backup.log                     # off-site sync log
BACKUP_TAG=manual-$(date -u +%H%M) bash scripts/ops/pg-backup.sh   # ad-hoc dump
sudo /usr/local/bin/naro-offsite-backup.sh                    # run the off-site sync now
cat /etc/cron.d/naro-pg-backup /etc/cron.d/naro-offsite-backup # schedules
```

Nginx

```
nginx -t                            # Test config syntax
systemctl reload nginx              # Reload config (no downtime)
systemctl restart nginx             # Full restart
systemctl status nginx              # Check status
cat /etc/nginx/sites-enabled/narofashion    # View current config
nano /etc/nginx/sites-available/narofashion # Edit config
```

PostgreSQL

```
sudo -u postgres psql                       # Open psql as superuser
sudo -u postgres psql -d naro_fashion       # Open psql to naro_fashion database
sudo -u postgres psql -d naro_fashion -c 'SELECT * FROM "Tenant";'   # Run a query
systemctl status postgresql                 # Check status
systemctl restart postgresql                # Restart
systemctl start postgresql                  # Start
```

Common SQL queries:

```
-- List all tenants
SELECT id, name, slug, domain, status FROM "Tenant";

-- List all admin users
SELECT id, email, "firstName", "lastName", role, "tenantId" FROM "AdminUser";

-- List platform admins
SELECT id, email, "firstName", "lastName" FROM "PlatformAdmin";

-- Check enabled modules for a tenant
SELECT tm.*, md.name FROM "TenantModule" tm
JOIN "ModuleDefinition" md ON md.code = tm."moduleCode"
WHERE tm."tenantId" = '<tenant-id>' AND tm.enabled = true;

-- Count records in key tables
SELECT
  (SELECT COUNT(*) FROM "Product") as products,
  (SELECT COUNT(*) FROM "Order") as orders,
  (SELECT COUNT(*) FROM "User") as customers,
  (SELECT COUNT(*) FROM "Rental") as rentals;
```

Certbot (SSL)

```
certbot certificates                        # List all certificates + expiry dates
certbot renew --dry-run                     # Test auto-renewal
certbot renew --force-renewal               # Force renew all certificates
certbot --nginx -d newdomain.com            # Add SSL for a new domain
```

UFW (Firewall)

```
ufw status                      # Show current rules
ufw allow OpenSSH               # Allow SSH
ufw allow 'Nginx Full'          # Allow HTTP + HTTPS
ufw enable                      # Enable firewall
ufw disable                     # Disable firewall (careful!)
```

System

```
free -h                          # Check RAM and swap usage
df -h                            # Check disk space
htop                             # Interactive process monitor
uptime                           # Server uptime and load
du -sh /var/www/naro-fashion     # Check app directory size
du -sh /var/backups/naro         # Check backup directory size
cat /etc/os-release              # Check OS version
node -v                          # Check Node.js version
pnpm -v                          # Check pnpm version
```

Prisma (Database) — always the local CLI, from `packages/database/`

```
cd /var/www/naro-fashion/packages/database

pnpm exec prisma generate               # Regenerate Prisma client
pnpm exec prisma migrate status         # Applied / pending / failed migrations
pnpm exec prisma migrate deploy         # Apply pending committed migrations (deploy.sh does
this)
pnpm exec prisma migrate resolve --rolled-back <name>   # Clear a failed migration (Part
15.4)
pnpm exec prisma migrate diff --from-schema-datasource prisma/schema.prisma \
  --to-schema-datamodel prisma/schema.prisma --script --exit-code   # Drift check (exit 0 =
none)
# NEVER in production: prisma db push (with or without --accept-data-loss), prisma migrate
dev/reset
```

Git

```
cd /var/www/naro-fashion

git status --short              # Must be clean (untracked .env files are git-ignored)
git log --oneline -3            # What the working tree is on
git log origin/prod --oneline -3   # Compare with what CI deploys (after git fetch origin
prod)
# Do NOT `git pull origin master` on the server — production tracks prod via deploy.sh
```

Infrastructure Reference

Current Production Setup

Component	Details
VPS Provider	Vultr (vultr.com)
Server Plan	vhf-1c-2gb (1 vCPU, 2GB RAM, 64GB NVMe SSD) — **shared with other applications**

Server Location	Frankfurt, Germany
Server IP	`80.240.30.107`
SSH host keys	ED25519 `SHA256:5/g02liZ1pxrwxbWOHTEGaOtUdrJrtB8uzL7C7bg42M`, ECDSA `SHA256:2AuvcjqqaC2wpZpD8LVeKiWNgmnS7HybLI+M93wUYxs`
OS	Ubuntu 24.04 LTS
Domain Registrar	Habari Node (habari.co.tz)
DNS Provider	Vultr DNS (ns1.vultr.com, ns2.vultr.com)
SSL	Let's Encrypt (auto-renews every 90 days)
Email	Brevo SMTP (smtp-relay.brevo.com, 300/day free)
Node.js	v22.x LTS (`.nvmrc`)
Package Manager	pnpm 10.12.1 (`packageManager` in `package.json`)
Process Manager	PM2 — `naro-api`, `naro-storefront`, `naro-admin` (cluster mode, 1 instance each)
Web Server	Nginx
Database	PostgreSQL 16, schema via committed Prisma migrations
App Directory	`/var/www/naro-fashion` (branch `prod`)
Local backups	`/var/backups/naro/postgres/` — nightly 03:15 UTC + pre-deploy, 30 days
Off-site backups	Cloudflare R2 (owner's account), nightly 04:00 UTC, `age`-encrypted; config in `/etc/naro-backup.env`
Backup alerting	Healthchecks.io project "Naro Fashion" (owner's account)
CI/CD	GitHub Actions `.github/workflows/deploy-prod.yml` (verify → deploy) on push to `prod`

Default Accounts (CHANGE PASSWORDS IMMEDIATELY AFTER FIRST LOGIN)

Login Type	Email	URL
Tenant Admin	`admin@narofashion.co.tz`	`https://admin.narofashion.co.tz/login`
Platform Admin	`platform@naro.co.tz`	`https://admin.narofashion.co.tz/platform-login`

The seeded default passwords are defined in `packages/database/prisma/migrate-to-multi-tenant.js` — they are public knowledge, so change them and enable 2FA on first login.

DNS Records (Vultr DNS)

Type	Name	Value	TTL
A	`@`	`80.240.30.107`	300
A	`www`	`80.240.30.107`	3600
A	`admin`	`80.240.30.107`	3600
A	`api`	`80.240.30.107`	3600
CNAME	`*`	`narofashion.co.tz`	300
NS		`ns1.vultr.com`	300
NS		`ns2.vultr.com`	300

Related Documents

- `docs/OPS/DEPLOY.md` — deploy + rollback runbook (short form of Part 11 / 15)
- `docs/OPS/MIGRATIONS.md` — migration workflow, baseline, failed migrations
- `docs/OPS/BACKUPS.md` — backup setup, restore, restore-test log
- `docs/OPS/MONITORING.md` — UptimeRobot, Healthchecks.io, Sentry setup
- `docs/OPERATIONS_RUNBOOK.md` , `docs/SECURITY.md` — incident response, threat model

Naro Fashion -- Multi-Tenant SaaS E-Commerce Platform Deployed on Vultr VPS, Frankfurt, Germany Last updated: October 2026 (Version 3.0)